

DEEP LEARNING FOR TIME SERIES



OUTLINE

- [Reminder] Neural networks 101
- Deep learning for time series:
 - Recurrent neural network (RNN)
 - Long short-term memory (LSTM)
 - Gated recurrent unit (GRU)
 - 1D convolutional neural network (1D-CNN)
- Transformers



OUTLINE

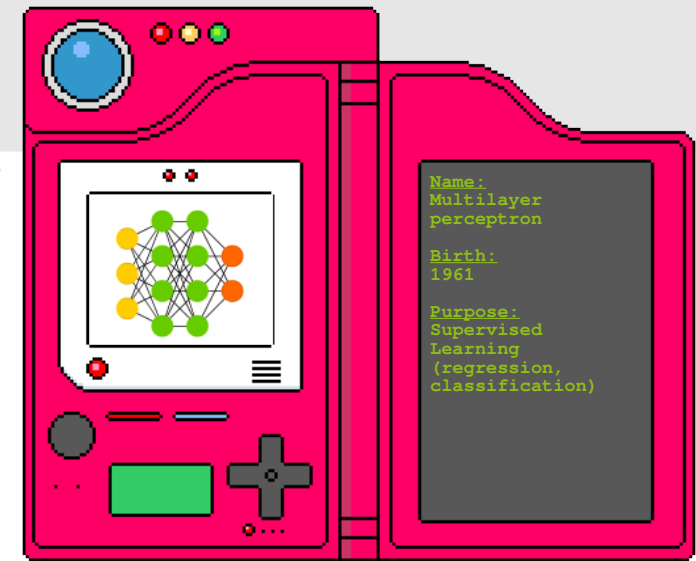
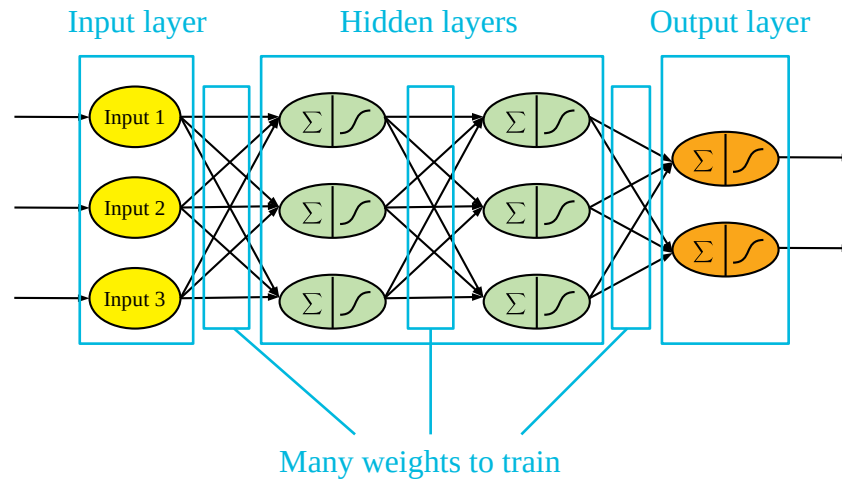
- [Reminder] Neural networks 101
- Deep learning for time series:
 - Recurrent neural network (RNN)
 - Long short-term memory (LSTM)
 - Gated recurrent unit (GRU)
 - 1D convolutional neural network (1D-CNN)
- Transformers



[Reminder] Neural networks 101

└ MLP (1/2)

<https://kamaleshjicse.files.wordpress.com/2018/11/pokedex-kamalesh.gif>

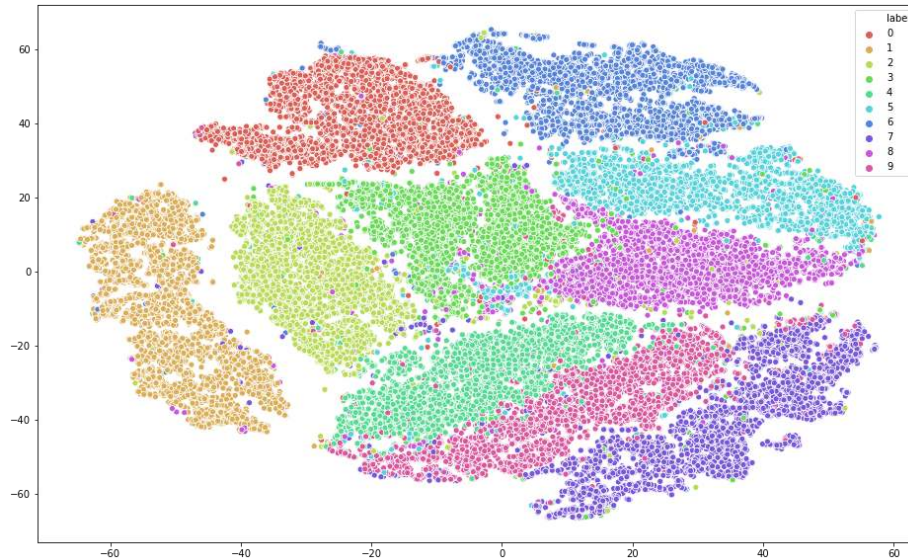


- Direct extension of the perceptron
- Also named « feedforward networks »
- Vertical and/or horizontal duplication of neurons
- Handles non-linearly separable datasets

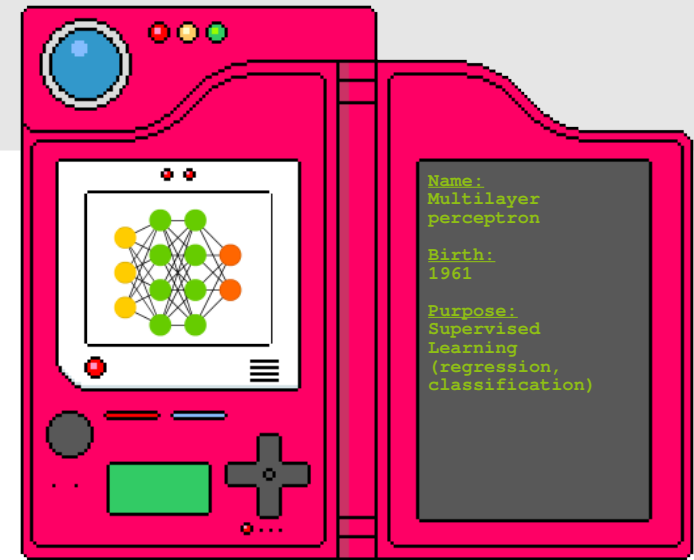
[Reminder] Neural networks 101

└ MLP (2/2)

<https://kamaleshjicse.files.wordpress.com/2018/11/pokedex-kamalesh.gif>
https://miro.medium.com/max/935/1*XFSxzNm2LWbVxKsHxkFwiQ.png



Example: MNIST 2D projection

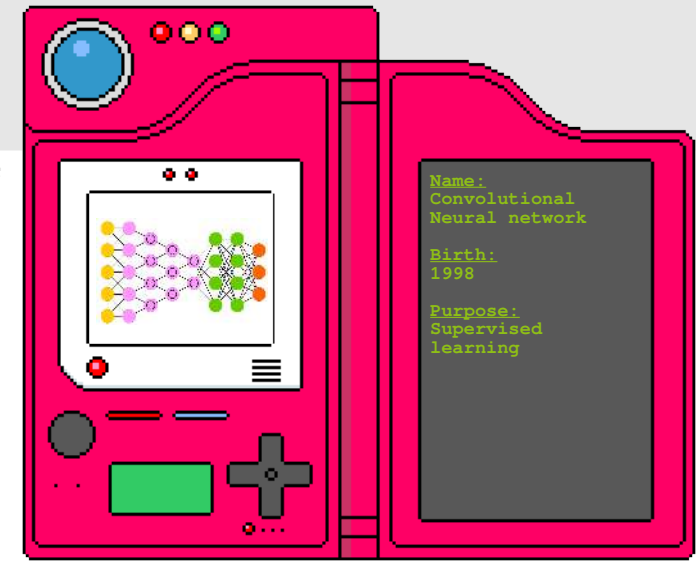
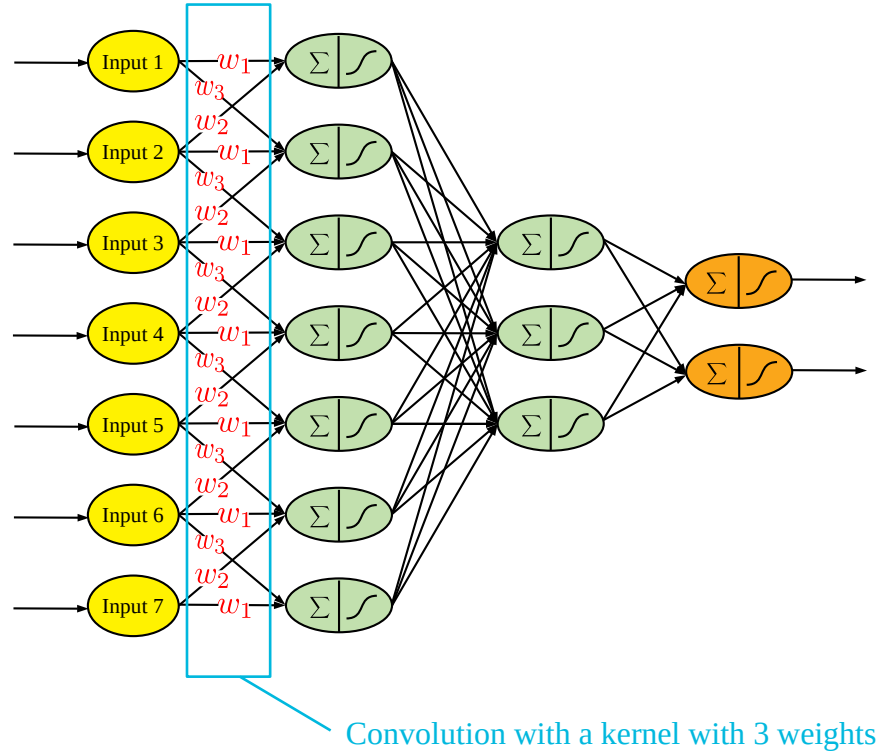


- Direct extension of the perceptron
- Also named « feedforward networks »
- Vertical and/or horizontal duplication of neurons
- Handles non-linearly separable datasets

[Reminder] Neural networks 101

└ CNN for images (1/3)

<https://kamaleshjicse.files.wordpress.com/2018/11/pokedex-kamalesh.gif>



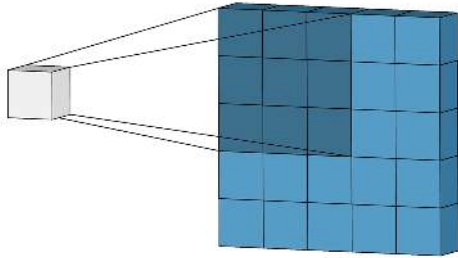
- Linear combination is replaced by convolution (captures locality of features)
- Some weights are shared (translation invariance)
- Ideal for data lying on Euclidean spaces
- Graph CNN extend this idea to irregular domains

[Reminder] Neural networks 101

└ CNN for images (2/3)

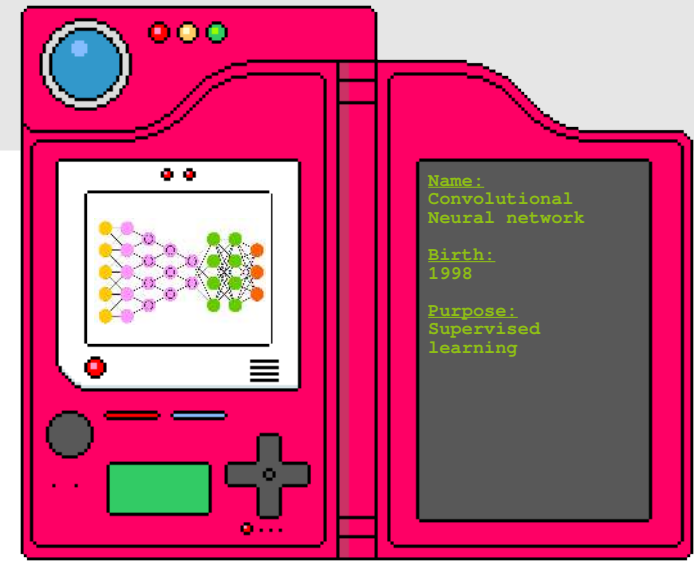
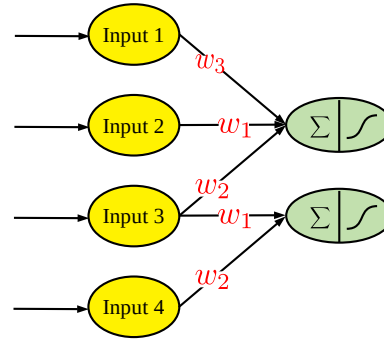
<https://kamaleshjicse.files.wordpress.com/2018/11/pokedex-kamalesh.gif>

<https://towardsdatascience.com/intuitively-understanding-convolutions-for-deep-learning-1f6f42faee1>



3_0	3_1	2_2	1	0
0_2	0_2	1_0	3	1
3_0	1_1	2_2	2	3
2	0	0	2	2
2	0	0	0	1

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0



- Linear combination is replaced by convolution (captures locality of features)
- Some weights are shared (translation invariance)
- Ideal for data lying on Euclidean spaces
- Graph CNN extend this idea to irregular domains

Deep learning for time series
Bastien Pasdeloup
14/11/2022

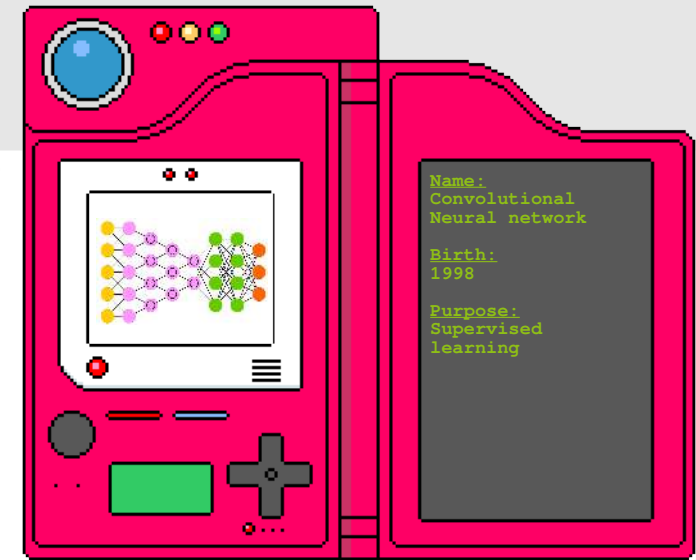
[Reminder] Neural networks 101

└ CNN for images (3/3)

<https://kamaleshjicse.files.wordpress.com/2018/11/pokedex-kamalesh.gif>
<https://jonathan-hui.medium.com/image-segmentation-with-mask-r-cnn-eb6d793272>



Example: Segmentation of images



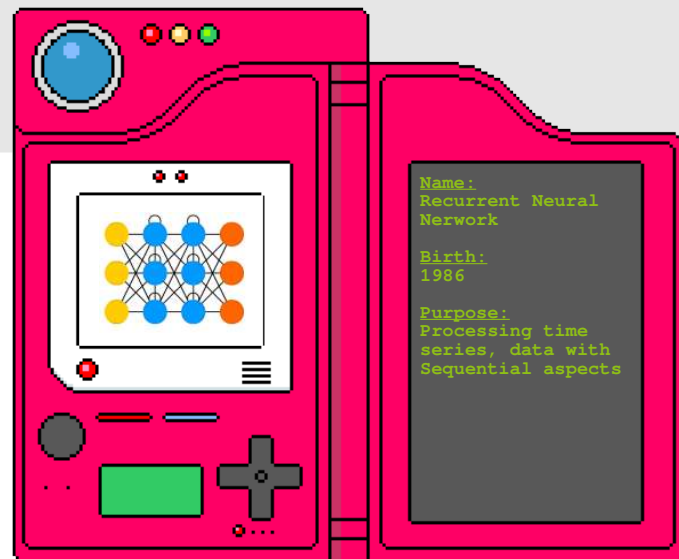
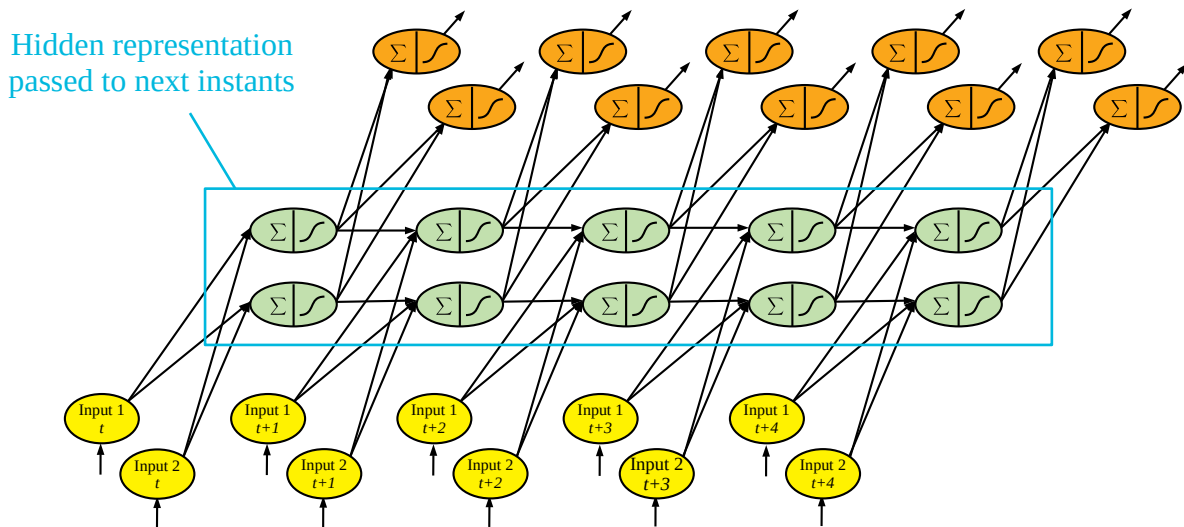
- Linear combination is replaced by convolution (captures locality of features)
- Some weights are shared (translation invariance)
- Ideal for data lying on Euclidean spaces
- Graph CNN extend this idea to irregular domains

[Reminder] Neural networks 101

└ RNN for time series (1/2)

<https://kamaleshjicse.files.wordpress.com/2018/11/pokedex-kamalesh.gif>

<https://medium.com/@jon.froiland/advanced-use-of-recurrent-neural-networks-part-1-91ff1b36af25>



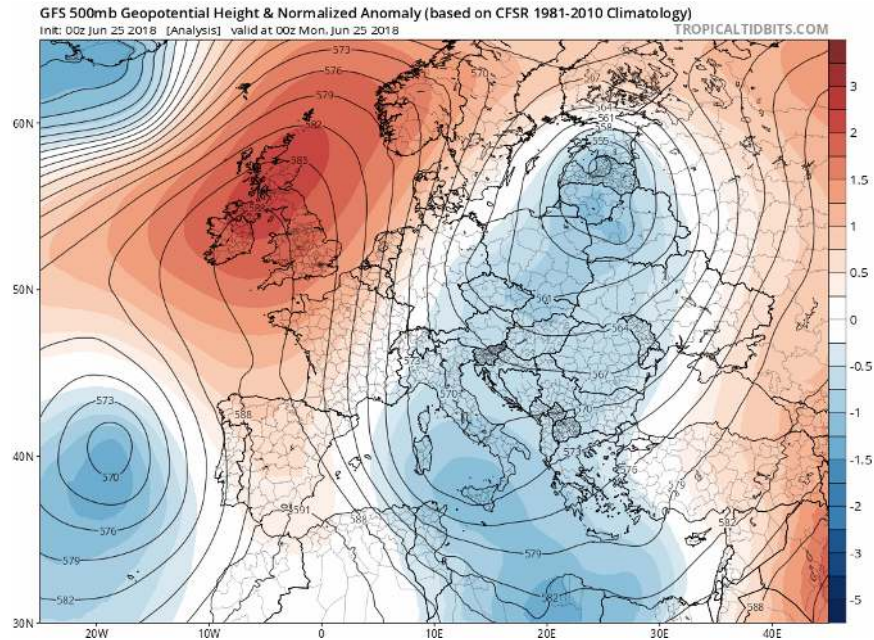
- Takes into account sequentiality of information (time series, texts...)
- Suffers a from gradient vanishing problem
- Base model for LSTMs or GRUs

[Reminder] Neural networks 101

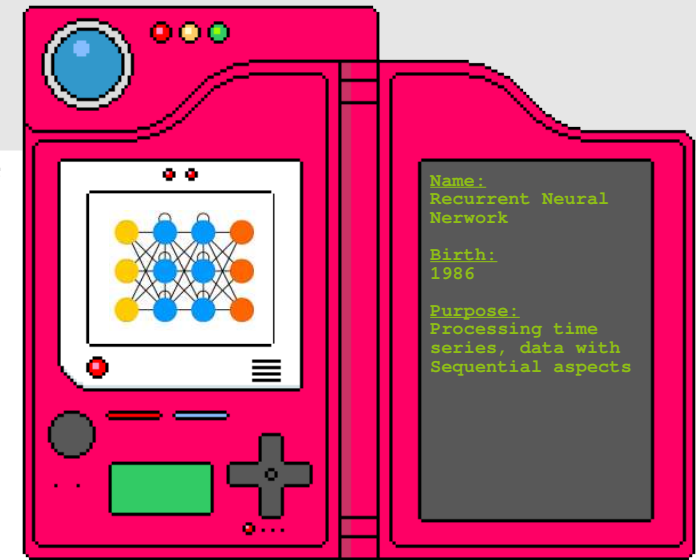
└ RNN for time series (2/2)

<https://kamaleshjicse.files.wordpress.com/2018/11/pokedex-kamalesh.gif>

<https://medium.com/@jon.froiland/advanced-use-of-recurrent-neural-networks-part-1-91ff1b36af25>



Example: Temperature forecasting



- Takes into account sequentiality of information (time series, texts...)
- Suffers a from gradient vanishing problem
- Base model for LSTMs or GRUs

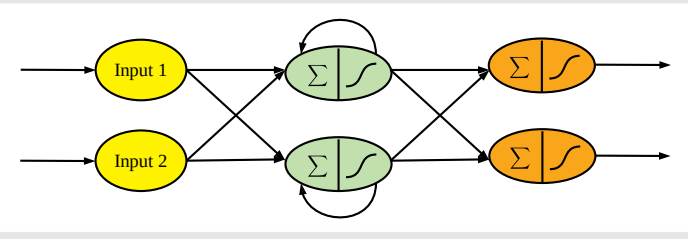
OUTLINE

- [Reminder] Neural networks 101
- Deep learning for time series:
 - Recurrent neural network (RNN)
 - Long short-term memory (LSTM)
 - Gated recurrent unit (GRU)
 - 1D convolutional neural network (1D-CNN)
- Transformers



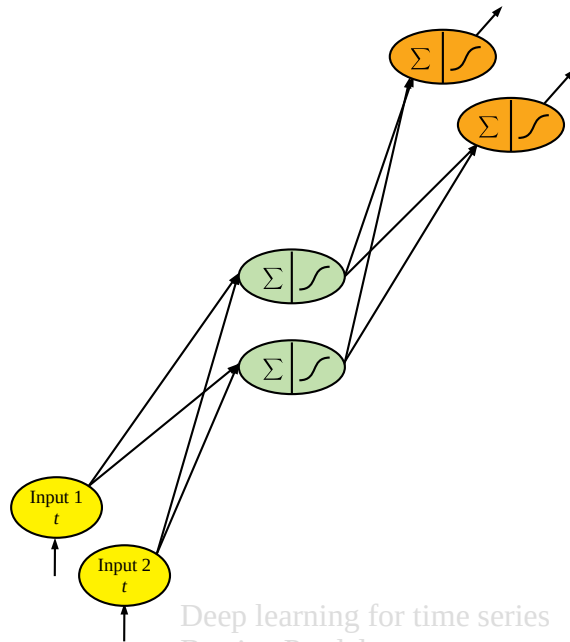
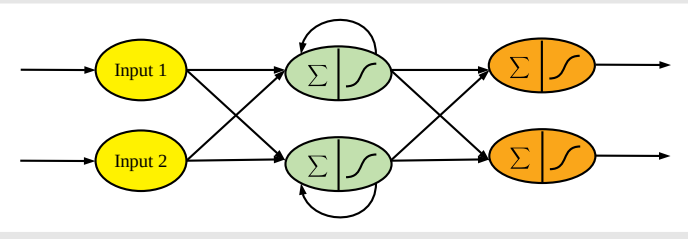
Deep learning for time series – RNN

└ Unfolding a RNN (1/6)



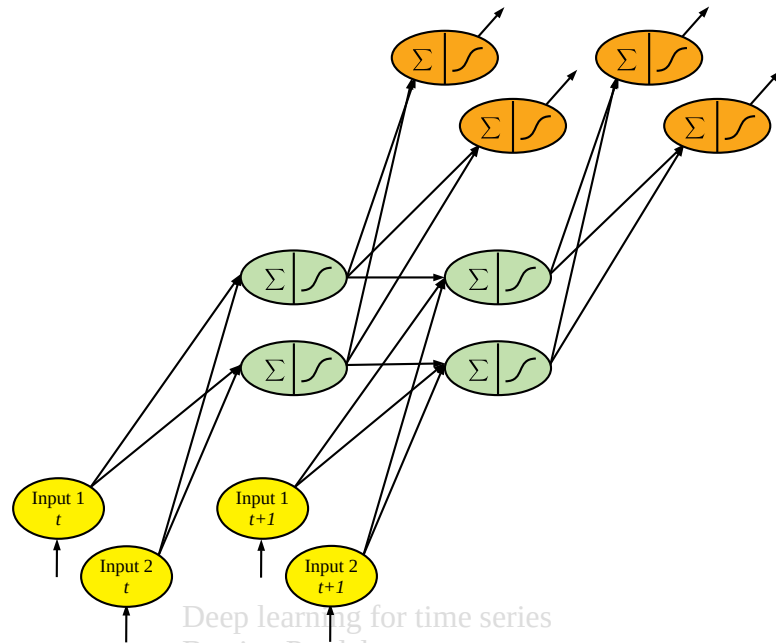
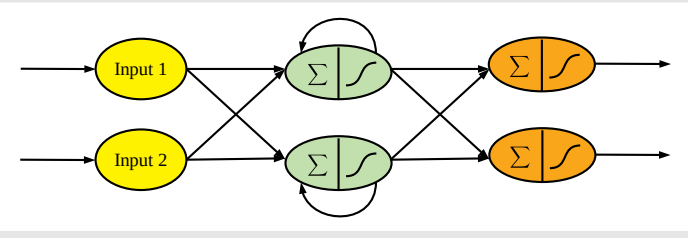
Deep learning for time series – RNN

└ Unfolding a RNN (2/6)



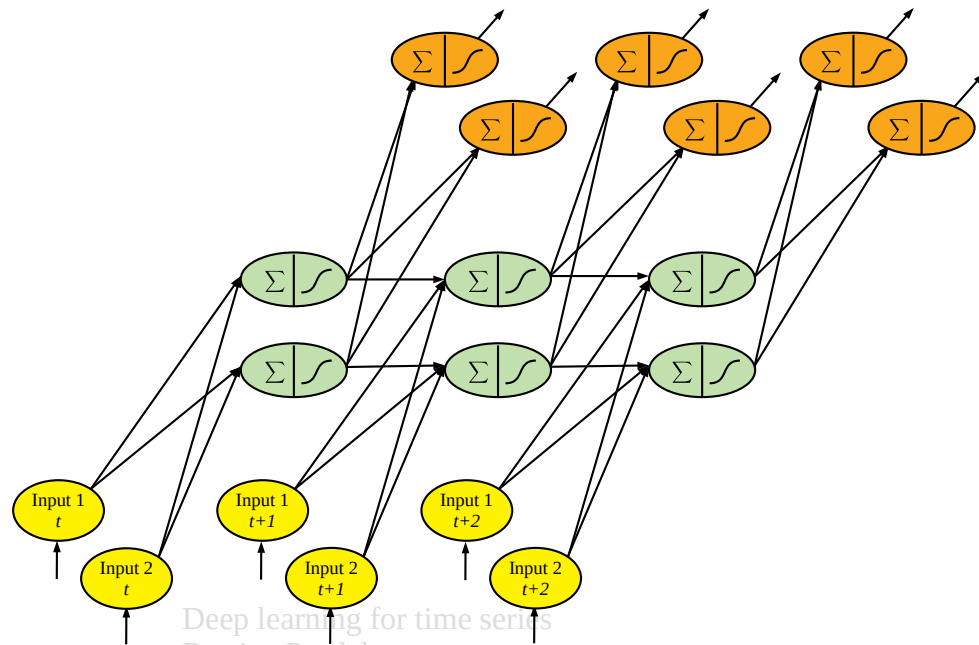
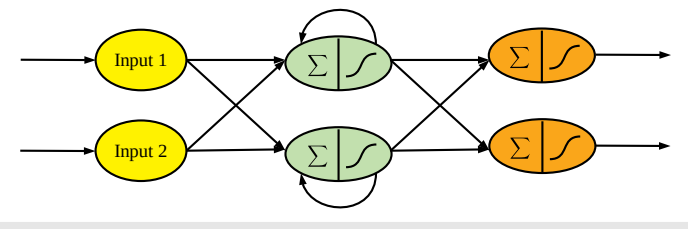
Deep learning for time series – RNN

└ Unfolding a RNN (3/6)



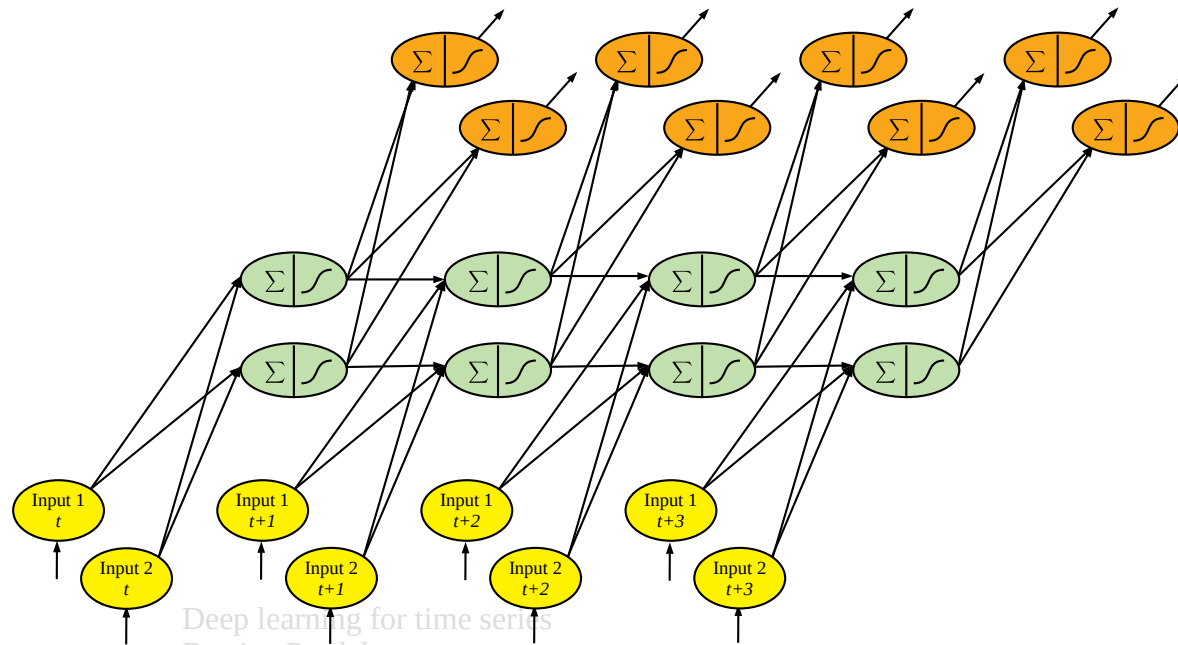
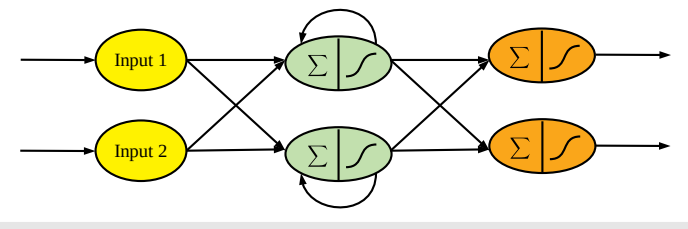
Deep learning for time series – RNN

└ Unfolding a RNN (4/6)



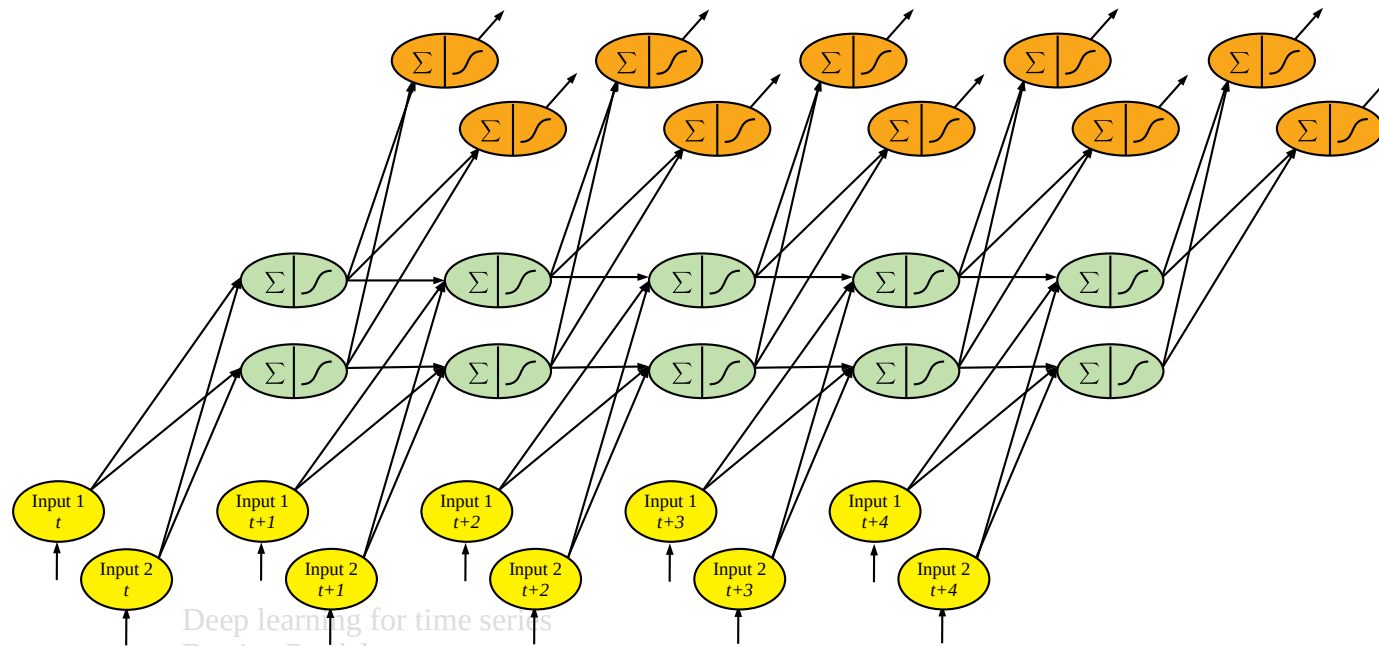
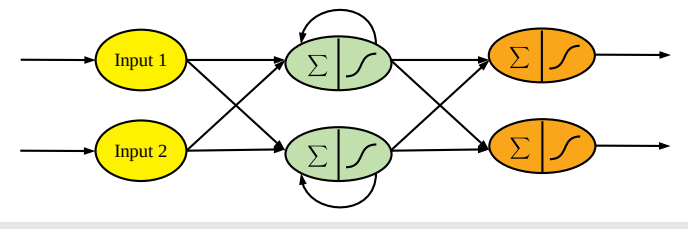
Deep learning for time series – RNN

└ Unfolding a RNN (5/6)



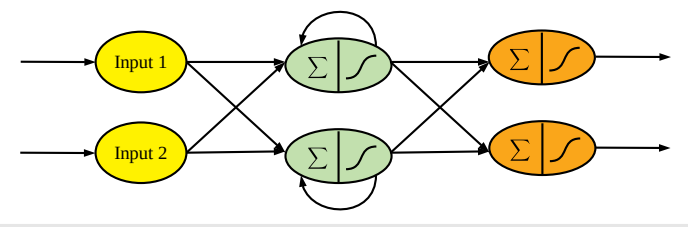
Deep learning for time series – RNN

└ Unfolding a RNN (6/6)



Deep learning for time series – RNN

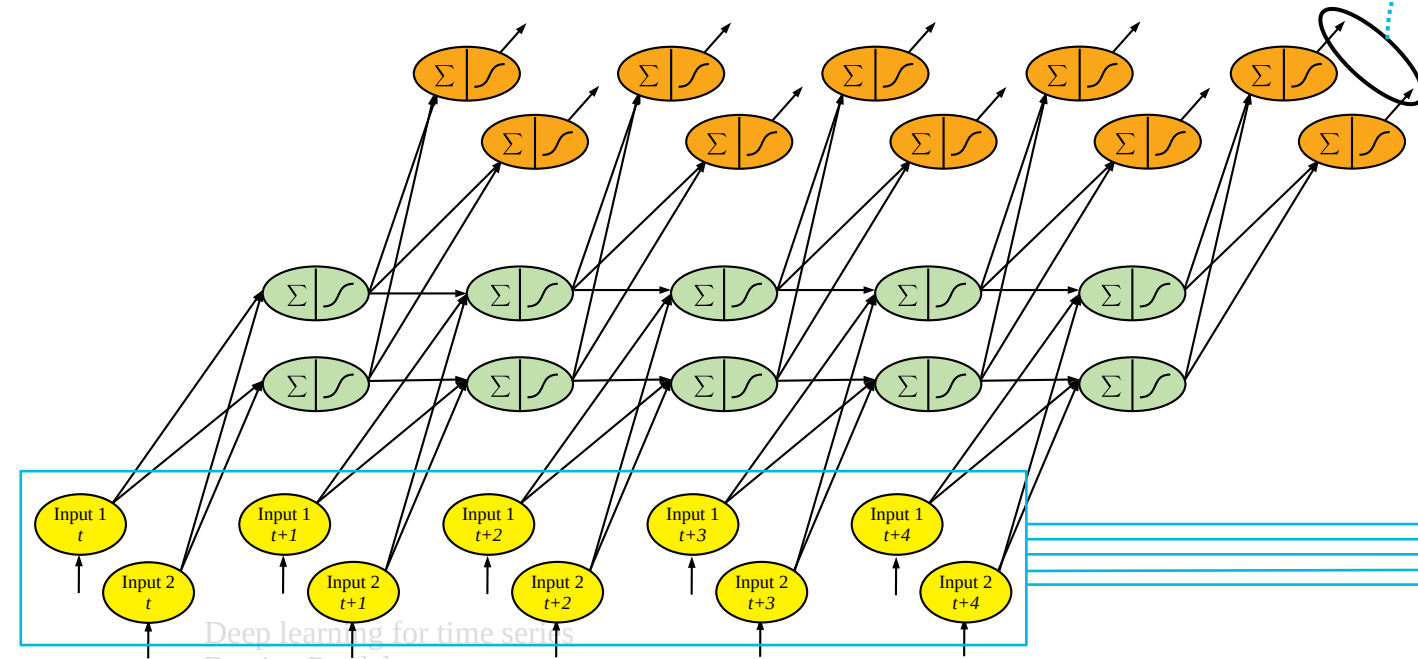
└ Application to text generation



<https://jghsenglish.edublogs.org/files/2015/02/Fahrenheit-451.pdf>

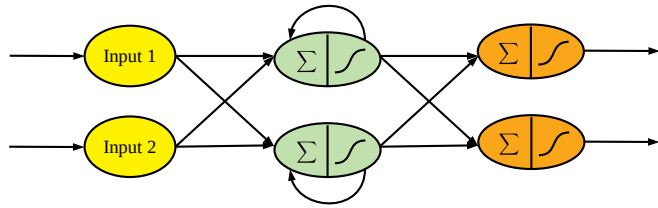
"Everyone

must leave something behind when he dies, my grandfather said. A child or a book or a painting or a house or a wall built or a pair of shoes made. Or a garden planted. Something your hand touched some way so your soul has somewhere to go when you die, and when people look at that tree or that flower you planted, you're there. It doesn't matter what you do, he said, so long as you change something from the way it was before you touched it into something that's like you after you take your hands away. The difference between the man who just cuts lawns and a real gardener is in the touching, he said. The lawn-cutter might just as well not have been there at all; the gardener will be there a lifetime."

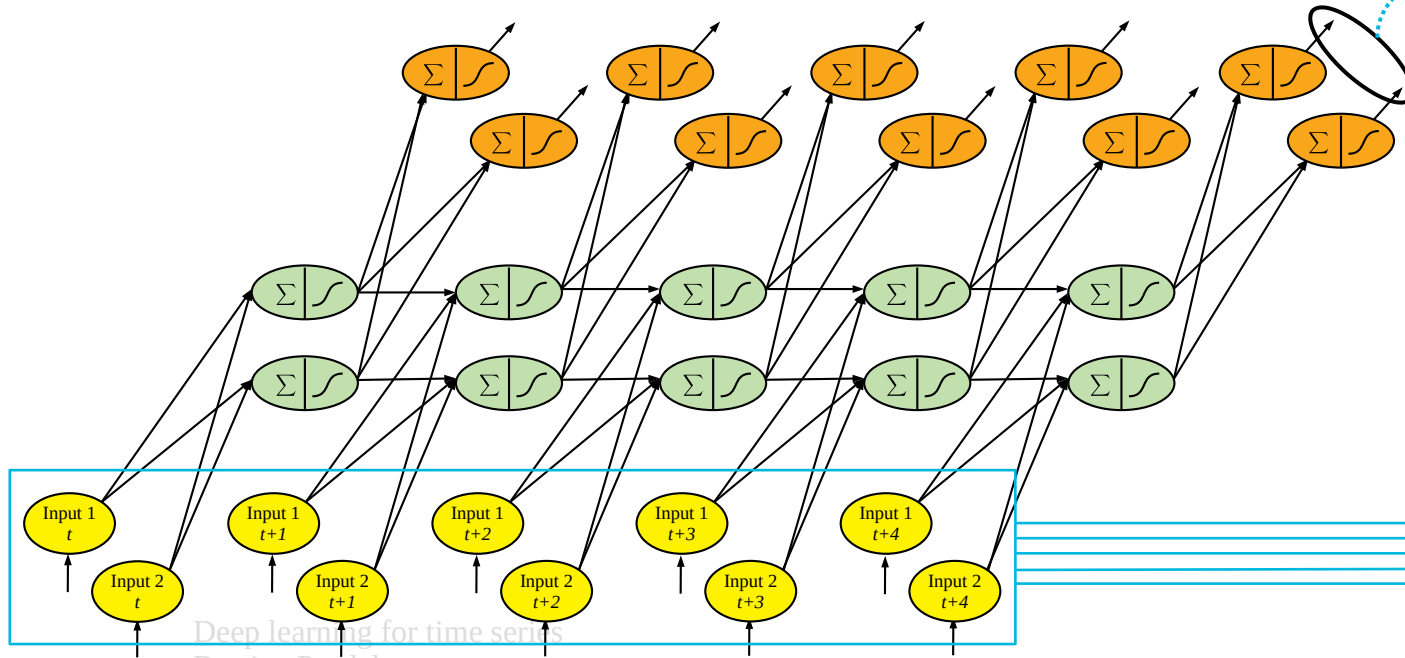
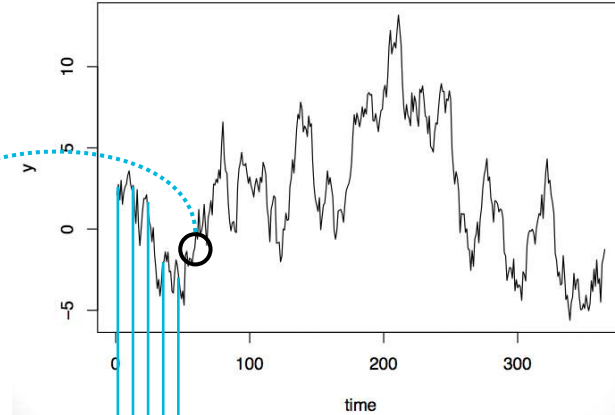


Deep learning for time series – RNN

└ Application to forecasting



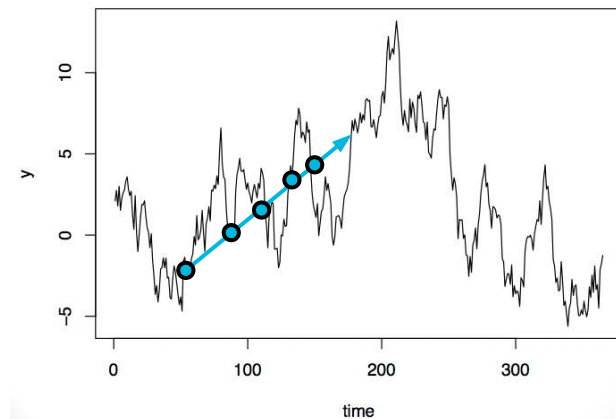
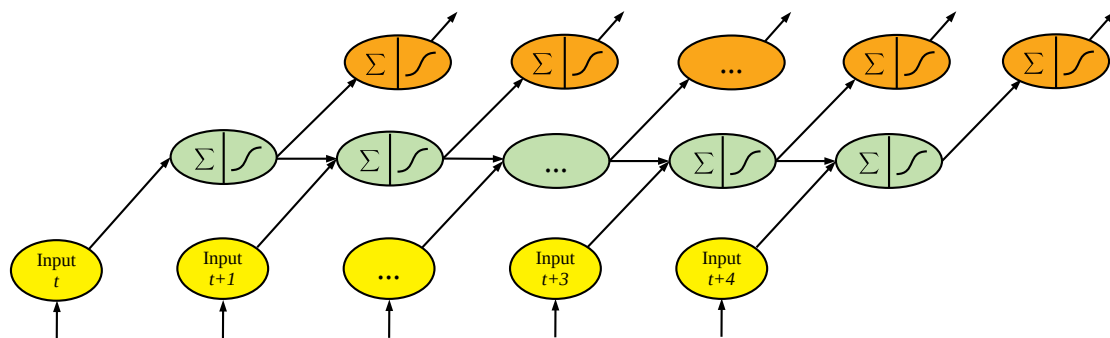
<https://simplystats.github.io/2016/05/05/timeseries-biomedical>



Deep learning for time series – RNN

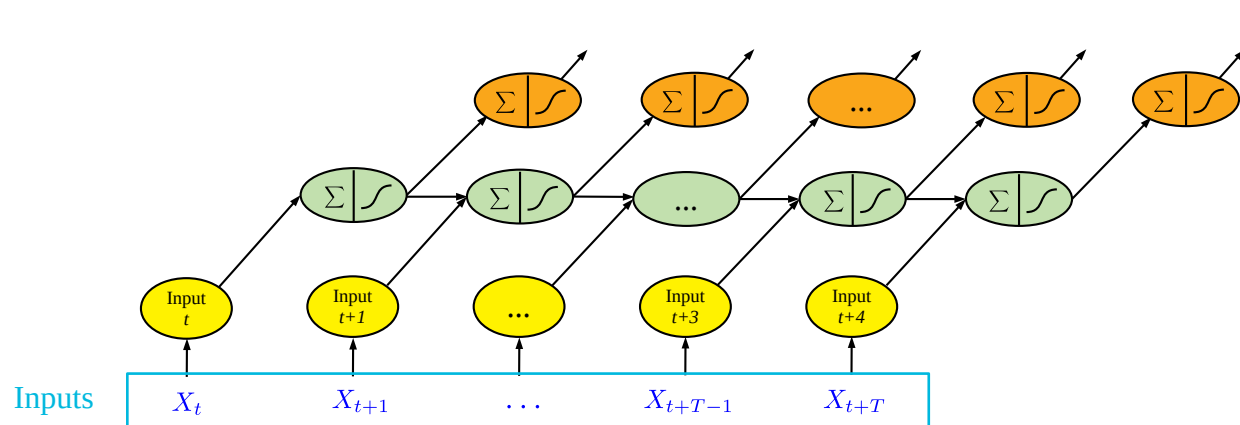
└ The gradient vanishing problem (1/13)

<https://simplystats.github.io/2016/05/05/timeseries-biomedical>

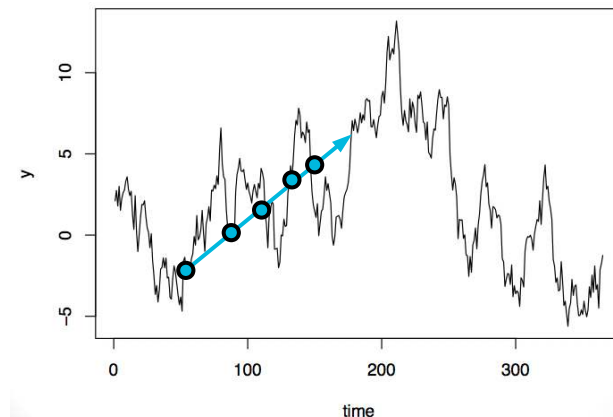


Deep learning for time series – RNN

└ The gradient vanishing problem (2/13)

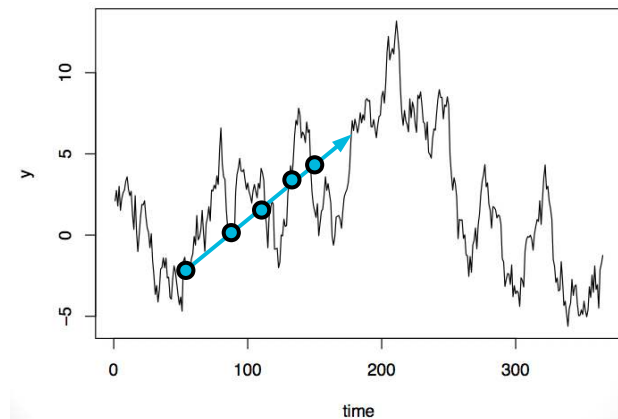
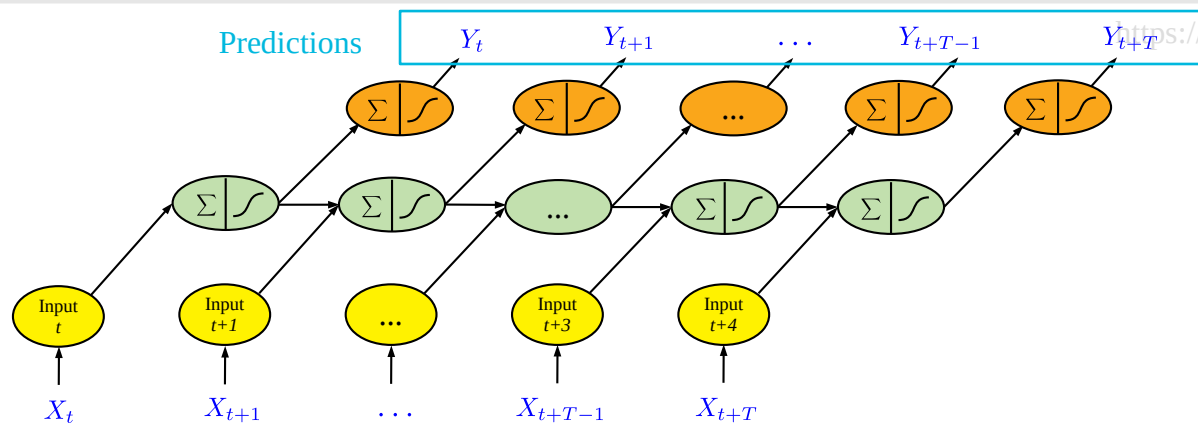


<https://simplystats.github.io/2016/05/05/timeseries-biomedical>



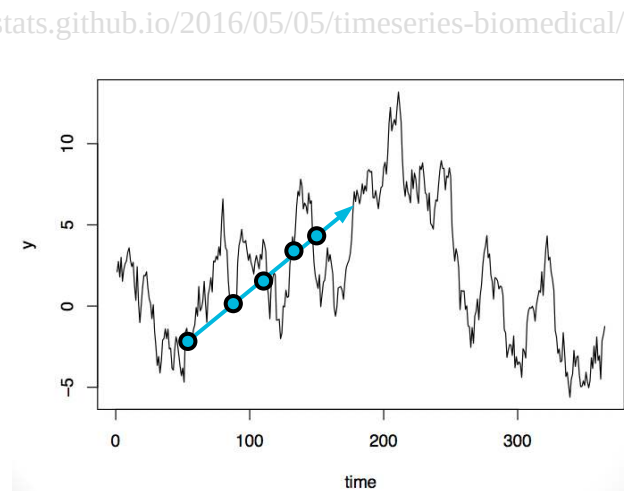
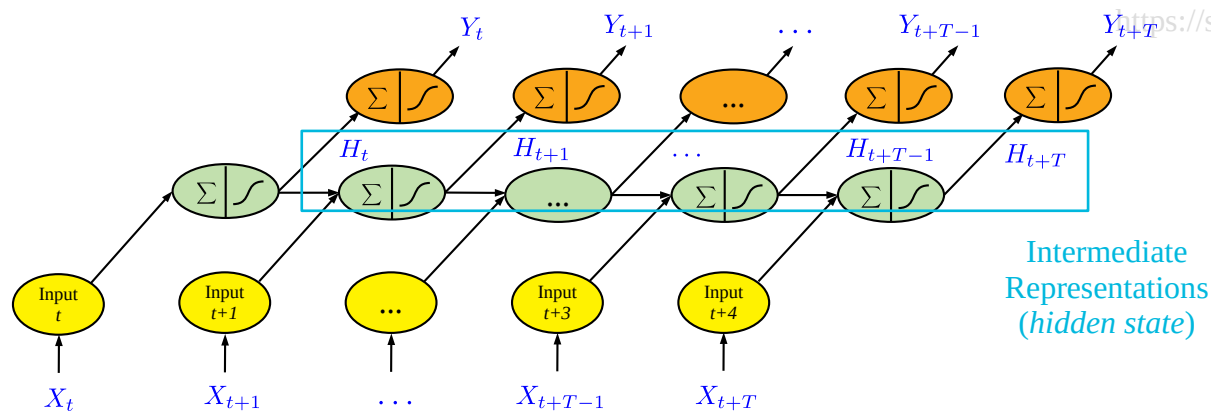
Deep learning for time series – RNN

└ The gradient vanishing problem (3/13)



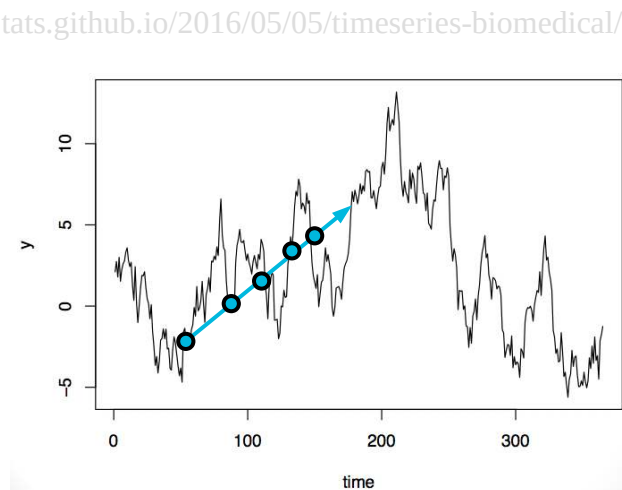
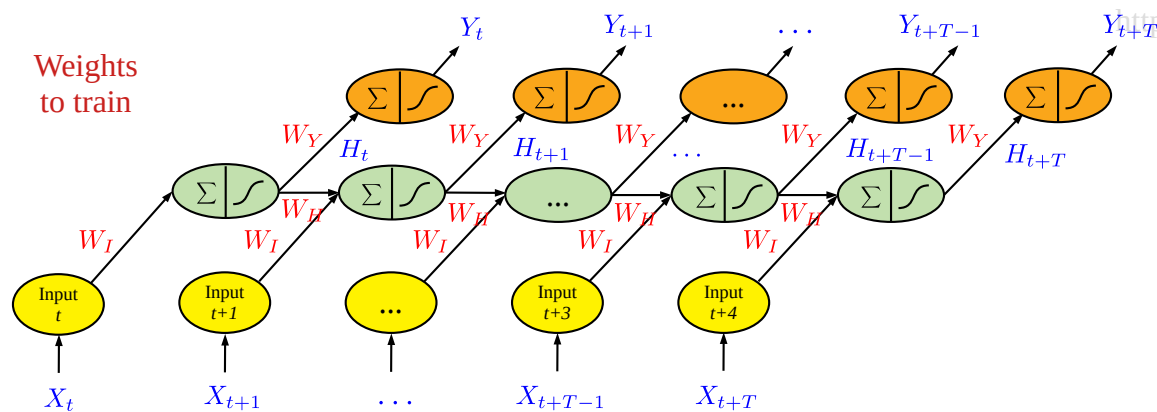
Deep learning for time series – RNN

└ The gradient vanishing problem (4/13)



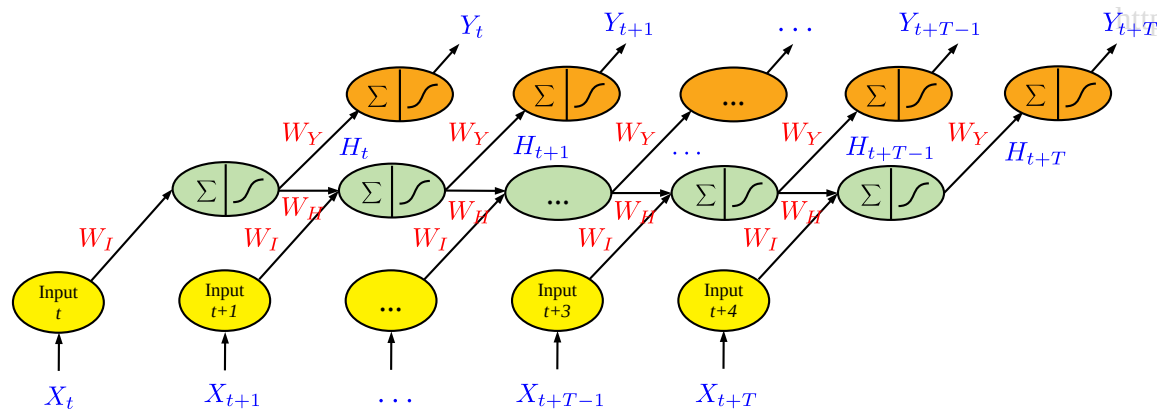
Deep learning for time series – RNN

└ The gradient vanishing problem (5/13)

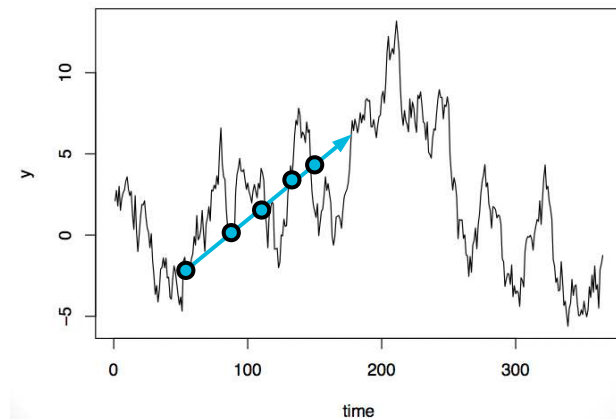


Deep learning for time series – RNN

└ The gradient vanishing problem (6/13)



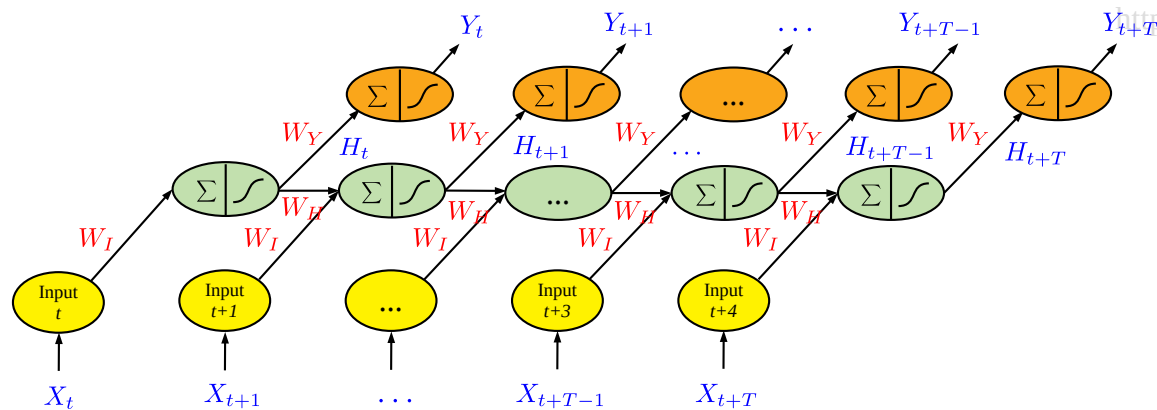
<https://simplystats.github.io/2016/05/05/timeseries-biomedical>



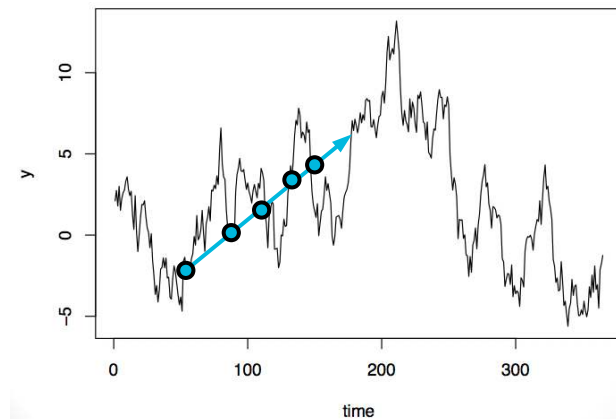
► Gradient computation is: $\frac{\partial E_{t+T}}{\partial W}$, with update: $W := W - \alpha \frac{\partial E_{t+T}}{\partial W}$

Deep learning for time series – RNN

└ The gradient vanishing problem (7/13)



<https://simplystats.github.io/2016/05/05/timeseries-biomedical>



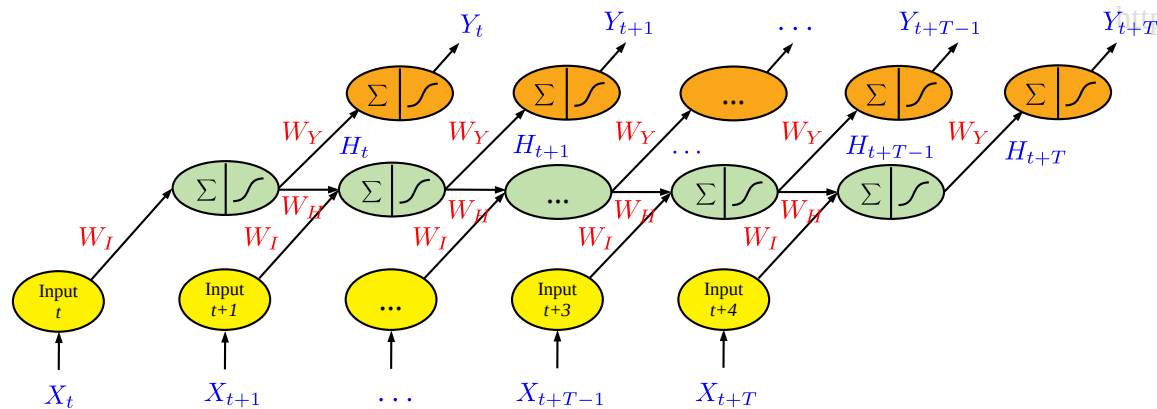
► Gradient computation is: $\frac{\partial E_{t+T}}{\partial W}$, with update: $W := W - \alpha \frac{\partial E_{t+T}}{\partial W}$

Final error
(e.g., MSE, cross-entropy...)

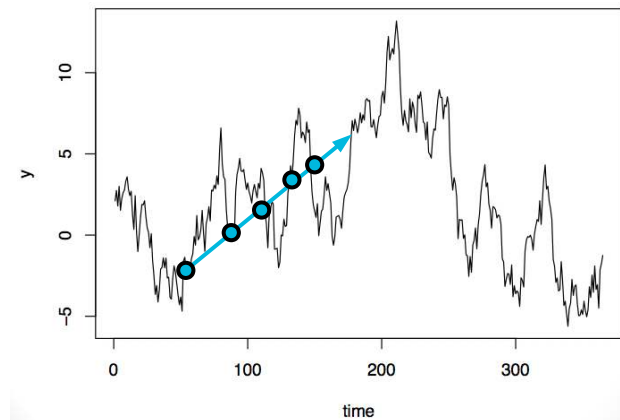
Let's study gradient with respect to $W = [W_I, W_H]$

Deep learning for time series – RNN

└ The gradient vanishing problem (8/13)



<https://simplystats.github.io/2016/05/05/timeseries-biomedical>



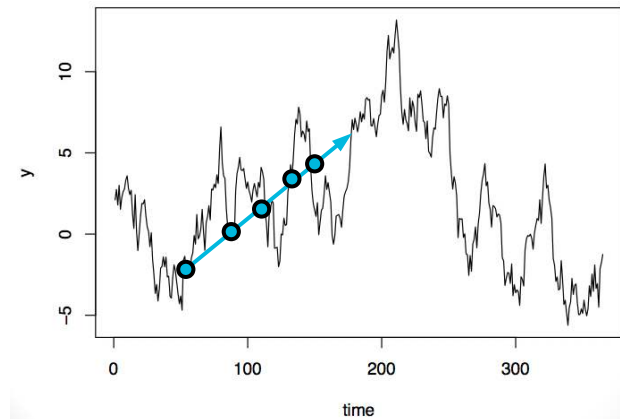
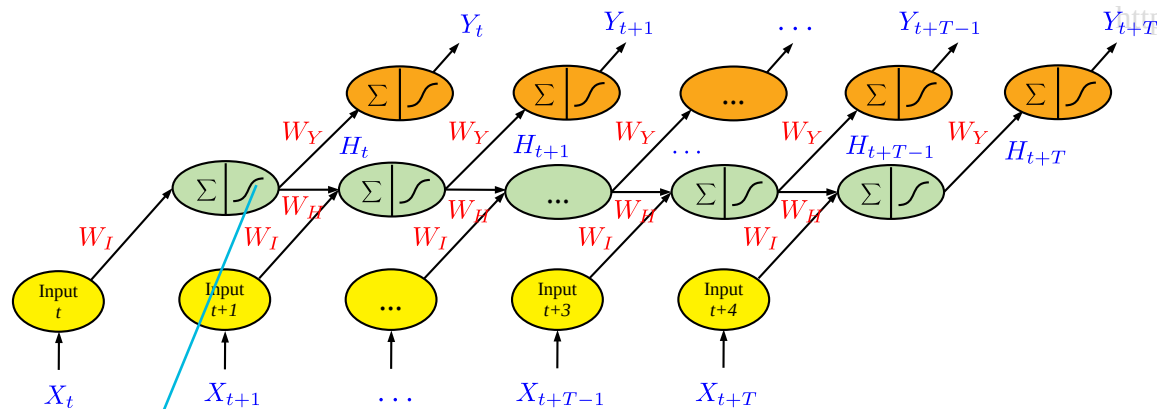
► Gradient computation is: $\frac{\partial E_{t+T}}{\partial W}$, with update: $W := W - \alpha \frac{\partial E_{t+T}}{\partial W}$

► We have the following equality: $\frac{\partial E_{t+T}}{\partial W} = \frac{\partial E_{t+T}}{\partial Y_{t+T}} \frac{\partial Y_{t+T}}{\partial H_{t+T}} \frac{\partial H_{t+T}}{\partial H_{t+T-1}} \frac{\partial H_{t+T-1}}{\partial H_{t+T-2}} \dots \frac{\partial H_{t+1}}{\partial H_t} \frac{\partial H_t}{\partial W} = \frac{\partial E_{t+T}}{\partial H_{t+T}} \frac{\partial Y_{t+T}}{\partial H_{t+T}} \left(\prod_{k=t+1}^{t+T} \frac{\partial H_k}{\partial H_{k-1}} \right) \frac{\partial H_t}{\partial W}$

Deep learning for time series – RNN

└ The gradient vanishing problem (9/13)

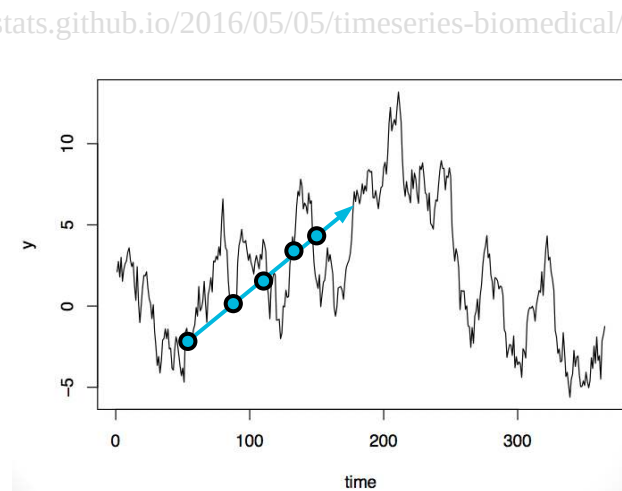
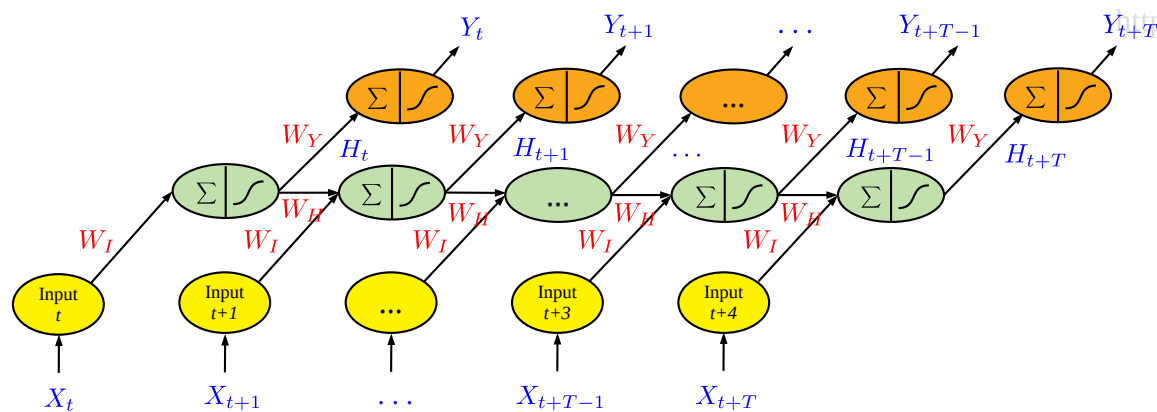
<https://simplystats.github.io/2016/05/05/timeseries-biomedical>



- Gradient computation is: $\frac{\partial E_{t+T}}{\partial W}$, with update: $W := W - \alpha \frac{\partial E_{t+T}}{\partial W}$
- We have the following equality:
$$\frac{\partial E_{t+T}}{\partial W} = \frac{\partial E_{t+T}}{\partial Y_{t+T}} \frac{\partial Y_{t+T}}{\partial H_{t+T}} \frac{\partial H_{t+T}}{\partial H_{t+T-1}} \frac{\partial H_{t+T-1}}{\partial H_{t+T-2}} \dots \frac{\partial H_{t+1}}{\partial H_t} \frac{\partial H_t}{\partial W} = \frac{\partial E_{t+T}}{\partial H_{t+T}} \frac{\partial Y_{t+T}}{\partial H_{t+T}} \left(\prod_{k=t+1}^{t+T} \frac{\partial H_k}{\partial H_{k-1}} \right) \frac{\partial H_t}{\partial W}$$
- Using a tanh nonlinearity, we have: $H_k = \tanh(W_H H_{k-1} + W_I I_k)$

Deep learning for time series – RNN

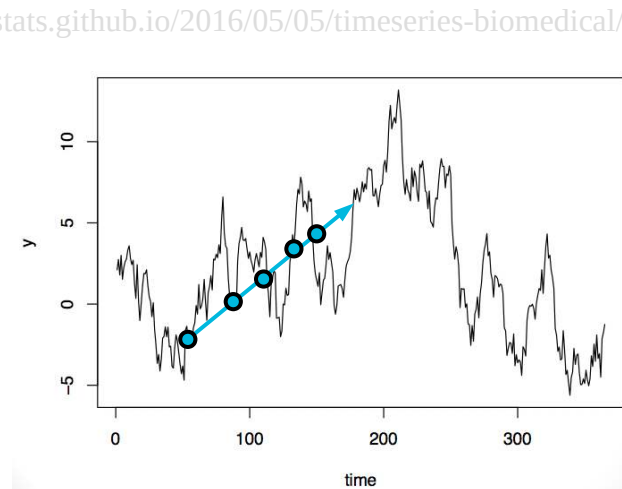
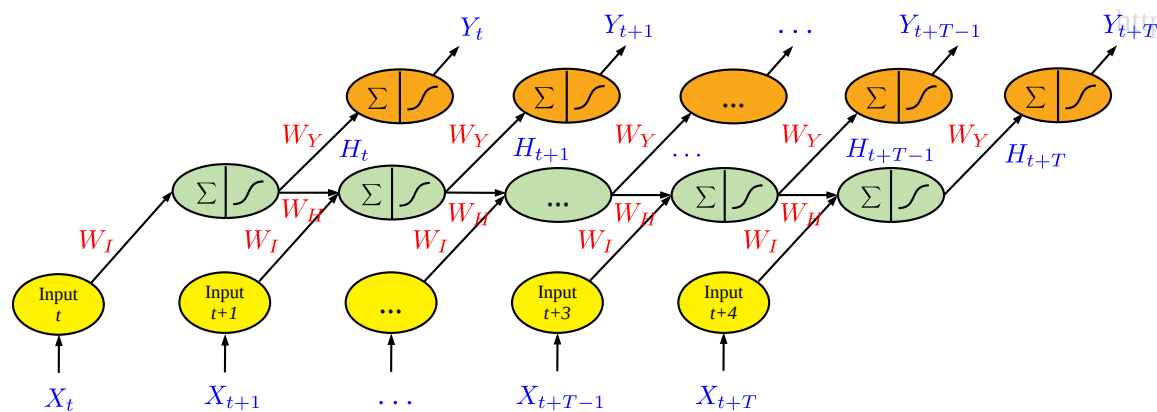
└ The gradient vanishing problem (10/13)



- Gradient computation is: $\frac{\partial E_{t+T}}{\partial W}$, with update: $W := W - \alpha \frac{\partial E_{t+T}}{\partial W}$
- We have the following equality:
$$\frac{\partial E_{t+T}}{\partial W} = \frac{\partial E_{t+T}}{\partial Y_{t+T}} \frac{\partial Y_{t+T}}{\partial H_{t+T}} \frac{\partial H_{t+T}}{\partial H_{t+T-1}} \frac{\partial H_{t+T-1}}{\partial H_{t+T-2}} \dots \frac{\partial H_{t+1}}{\partial H_t} \frac{\partial H_t}{\partial W} = \frac{\partial E_{t+T}}{\partial H_{t+T}} \frac{\partial Y_{t+T}}{\partial H_{t+T}} \left(\prod_{k=t+1}^{t+T} \frac{\partial H_k}{\partial H_{k-1}} \right) \frac{\partial H_t}{\partial W}$$
- Using a $\tanh$ nonlinearity, we have: $H_k = \tanh(W_H H_{k-1} + W_I I_k)$
- Derivating H_k , we get:
$$\frac{\partial H_k}{\partial H_{k-1}} = \tanh'(W_H H_{k-1} + W_I I_k) \frac{d}{dH_{k-1}} [W_H H_{k-1} + W_I I_k] = (1 - \tanh^2(W_H H_{k-1} + W_I I_k)) W_H$$

Deep learning for time series – RNN

└ The gradient vanishing problem (11/13)

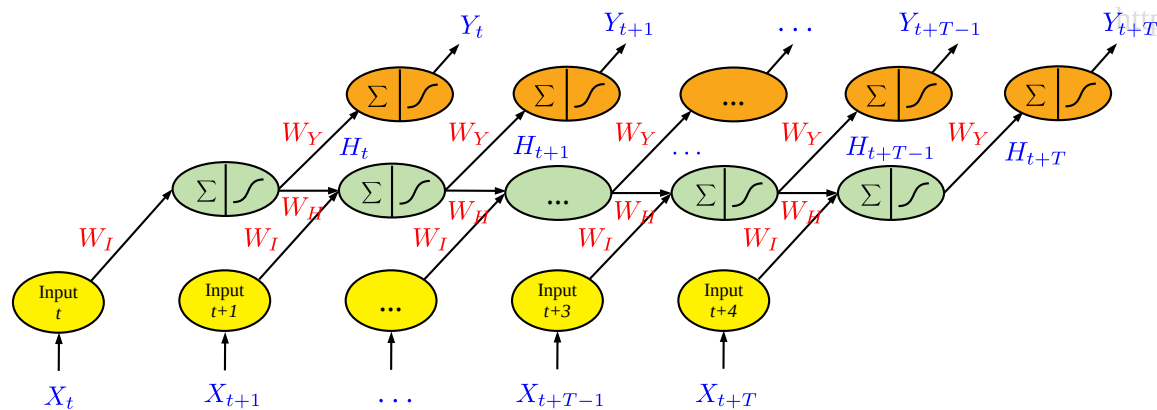


- Gradient computation is: $\frac{\partial E_{t+T}}{\partial W}$, with update: $W := W - \alpha \frac{\partial E_{t+T}}{\partial W}$
- We have the following equality:
$$\frac{\partial E_{t+T}}{\partial W} = \frac{\partial E_{t+T}}{\partial Y_{t+T}} \frac{\partial Y_{t+T}}{\partial H_{t+T}} \frac{\partial H_{t+T}}{\partial H_{t+T-1}} \frac{\partial H_{t+T-1}}{\partial H_{t+T-2}} \dots \frac{\partial H_{t+1}}{\partial H_t} \frac{\partial H_t}{\partial W} = \frac{\partial E_{t+T}}{\partial H_{t+T}} \frac{\partial Y_{t+T}}{\partial H_{t+T}} \left(\prod_{k=t+1}^{t+T} \frac{\partial H_k}{\partial H_{k-1}} \right) \frac{\partial H_t}{\partial W}$$
- Using a $\tanh$ nonlinearity, we have: $H_k = \tanh(W_H H_{k-1} + W_I I_k)$
- Derivating H_k , we get:
$$\frac{\partial H_k}{\partial H_{k-1}} = \tanh'(W_H H_{k-1} + W_I I_k) \frac{d}{dH_{k-1}} [W_H H_{k-1} + W_I I_k] = (1 - \tanh^2(W_H H_{k-1} + W_I I_k)) W_H$$

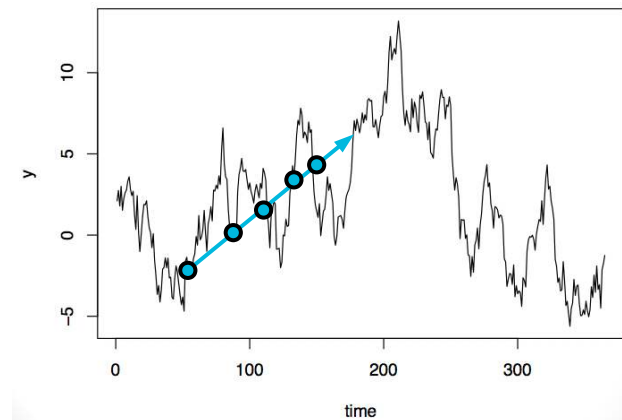
Plug here

Deep learning for time series – RNN

└ The gradient vanishing problem (12/13)



<https://simplystats.github.io/2016/05/05/timeseries-biomedical>



► Gradient computation is: $\frac{\partial E_{t+T}}{\partial W}$, with update: $W := W - \alpha \frac{\partial E_{t+T}}{\partial W}$

► We have the following equality: $\frac{\partial E_{t+T}}{\partial W} = \frac{\partial E_{t+T}}{\partial Y_{t+T}} \frac{\partial Y_{t+T}}{\partial H_{t+T}} \frac{\partial H_{t+T}}{\partial H_{t+T-1}} \frac{\partial H_{t+T-1}}{\partial H_{t+T-2}} \dots \frac{\partial H_{t+1}}{\partial H_t} \frac{\partial H_t}{\partial W} = \frac{\partial E_{t+T}}{\partial H_{t+T}} \frac{\partial Y_{t+T}}{\partial H_{t+T}} \left(\prod_{k=t+1}^{t+T} \frac{\partial H_k}{\partial H_{k-1}} \right) \frac{\partial H_t}{\partial W}$

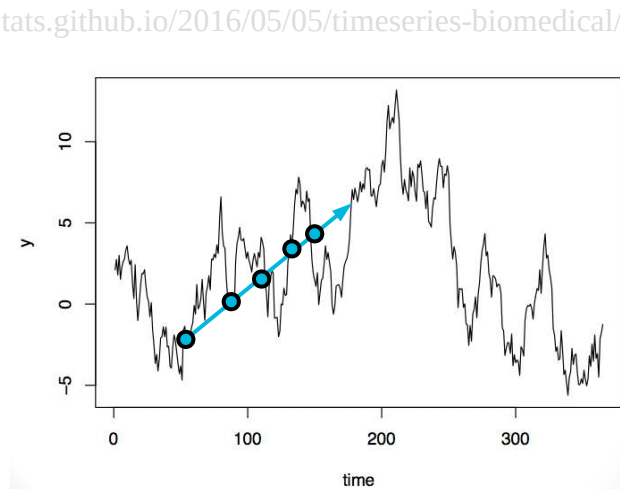
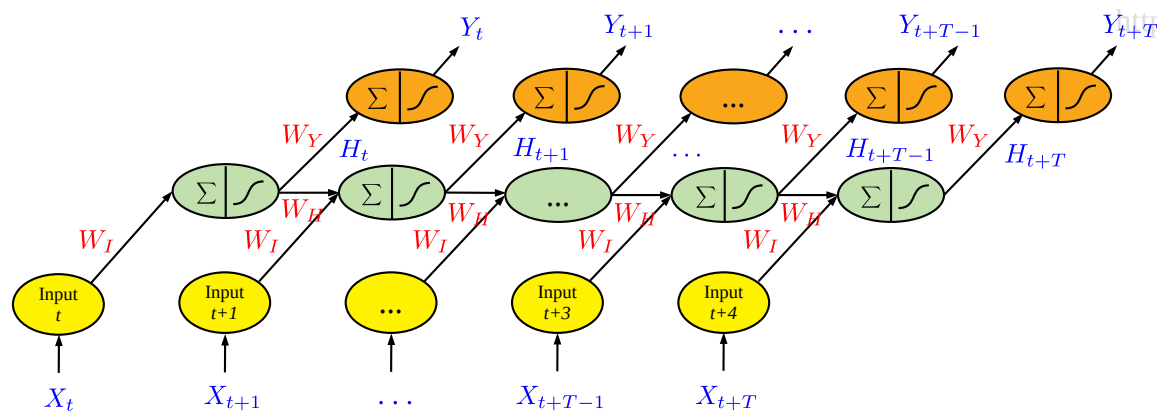
→ 0

► Using a $\tanh$ nonlinearity, we have: $H_k = \tanh(W_H H_{k-1} + W_I I_k)$

► Derivating H_k , we get: $\frac{\partial H_k}{\partial H_{k-1}} = \tanh'(W_H H_{k-1} + W_I I_k) \frac{d}{dH_{k-1}} [W_H H_{k-1} + W_I I_k] = (1 - \tanh^2(W_H H_{k-1} + W_I I_k)) W_H$

Deep learning for time series – RNN

└ The gradient vanishing problem (13/13)



- Gradient computation is: $\frac{\partial E_{t+T}}{\partial W}$, with update: $W := W - \alpha \frac{\partial E_{t+T}}{\partial W}$
- We have the following equality:
$$\frac{\partial E_{t+T}}{\partial W} = \frac{\partial E_{t+T}}{\partial Y_{t+T}} \frac{\partial Y_{t+T}}{\partial H_{t+T}} \frac{\partial H_{t+T}}{\partial H_{t+T-1}} \frac{\partial H_{t+T-1}}{\partial H_{t+T-2}} \dots \frac{\partial H_{t+1}}{\partial H_t} \frac{\partial H_t}{\partial W} = \frac{\partial E_{t+T}}{\partial H_{t+T}} \frac{\partial Y_{t+T}}{\partial H_{t+T}} \left(\prod_{k=t+1}^{t+T} \frac{\partial H_k}{\partial H_{k-1}} \right) \frac{\partial H_t}{\partial W}$$
- Using a $\tanh$ nonlinearity, we have: $H_k = \tanh(W_H H_{k-1} + W_I I_k)$
- Derivating H_k , we get:
$$\frac{\partial H_k}{\partial H_{k-1}} = \tanh'(W_H H_{k-1} + W_I I_k) \frac{d}{dH_{k-1}} [W_H H_{k-1} + W_I I_k] = (1 - \tanh^2(W_H H_{k-1} + W_I I_k)) W_H$$
- Conclusion: $\frac{\partial E_{t+T}}{\partial W} \rightarrow 0$, therefore: $W := W - \alpha \frac{\partial E_{t+T}}{\partial W} \approx W$, and nothing is retained across time :(

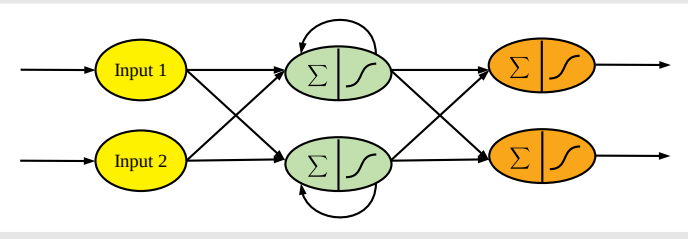
OUTLINE

- [Reminder] Neural networks 101
- Deep learning for time series:
 - Recurrent neural network (RNN)
 - Long short-term memory (LSTM)
 - Gated recurrent unit (GRU)
 - 1D convolutional neural network (1D-CNN)
- Transformers



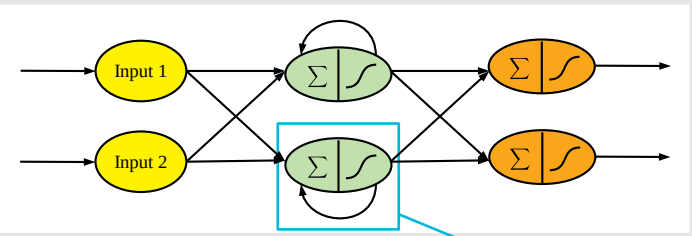
Deep learning for time series – LSTM

└ Anatomy of a LSTM (1/8)

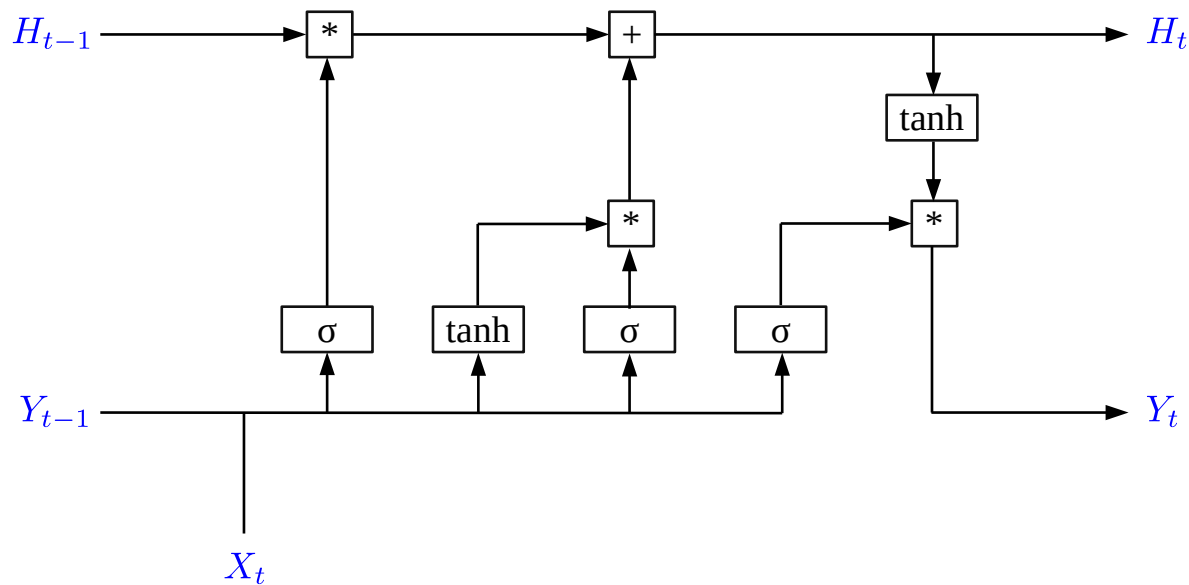


Deep learning for time series – LSTM

└ Anatomy of a LSTM (2/8)

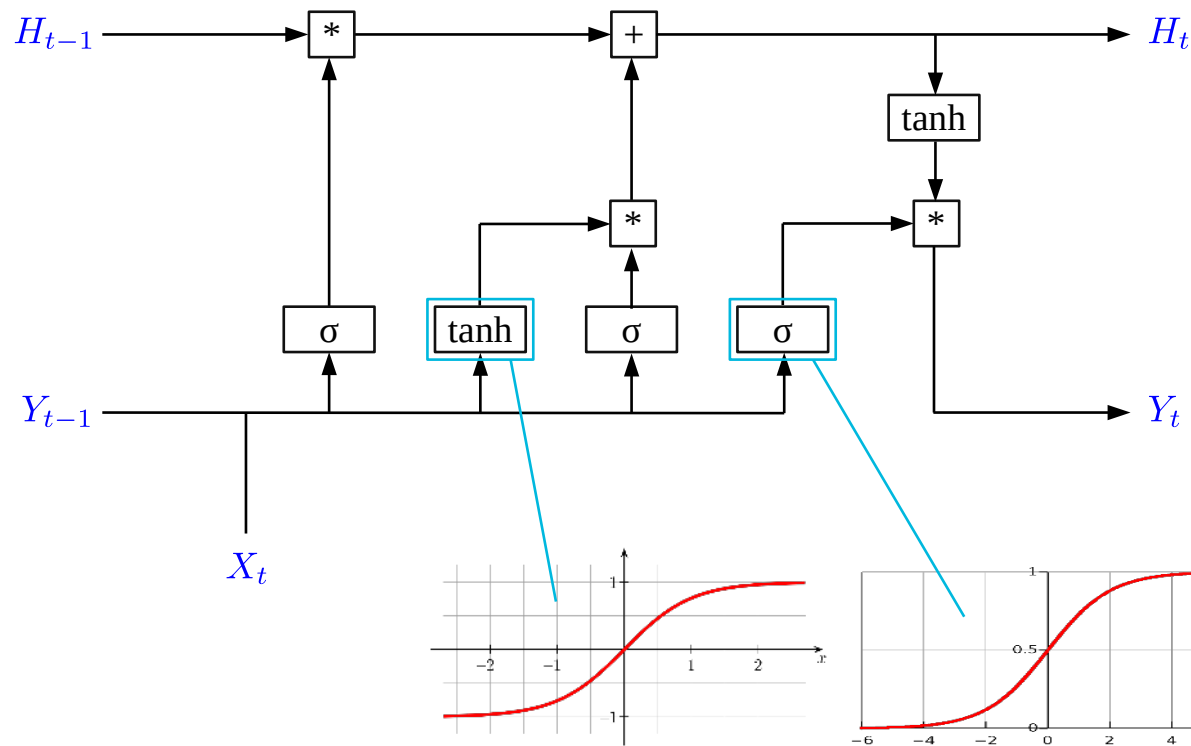
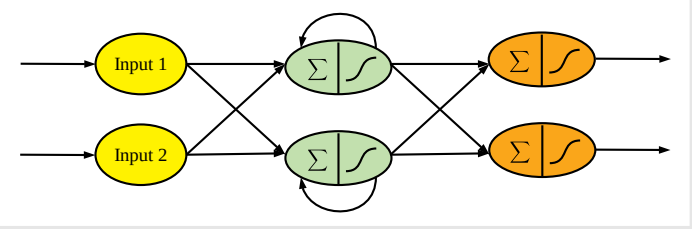


Replace with something
more complex
(we call it a cell)



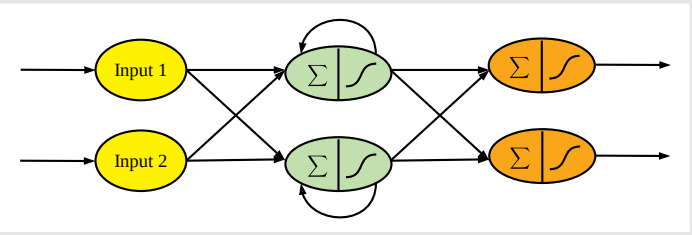
Deep learning for time series – LSTM

└ Anatomy of a LSTM (3/8)

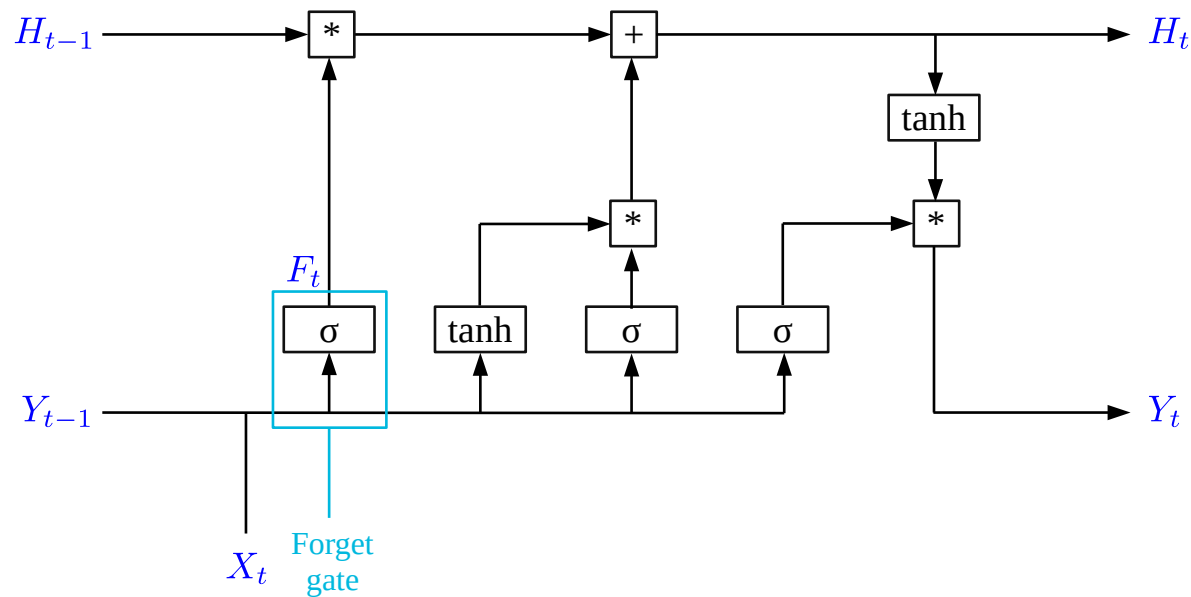


Deep learning for time series – LSTM

└ Anatomy of a LSTM (4/8)

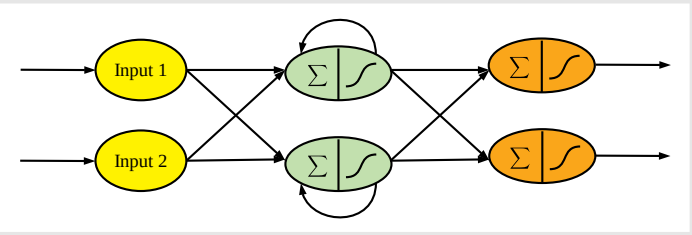


$$F_t = \sigma(W_{X_F} X_t + W_{Y_F} Y_{t-1} + B_F)$$



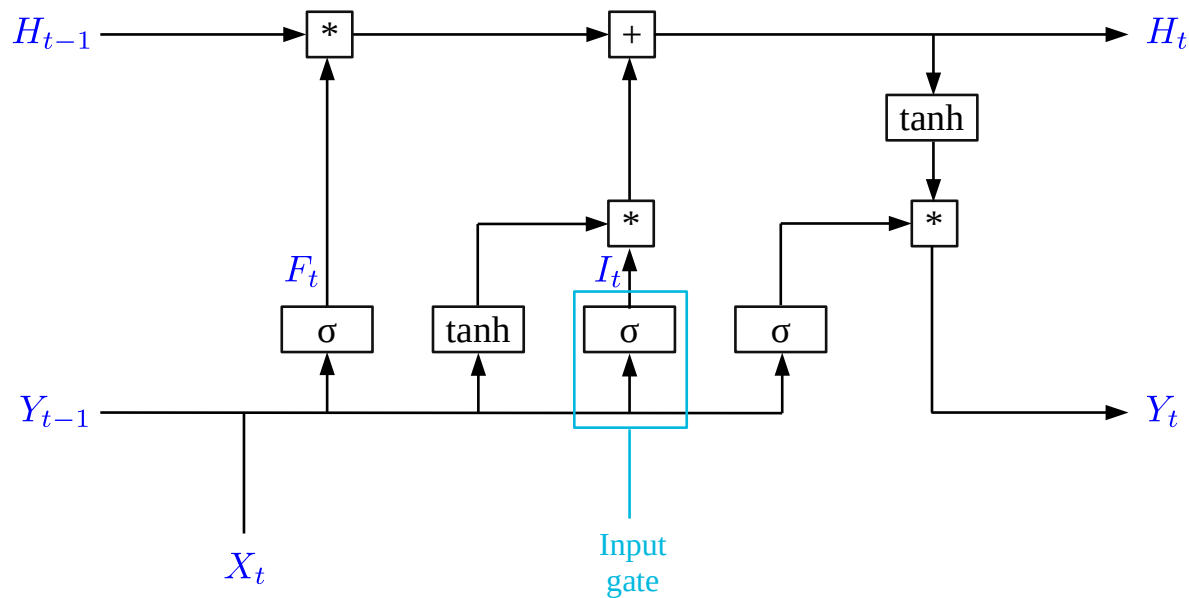
Deep learning for time series – LSTM

└ Anatomy of a LSTM (5/8)



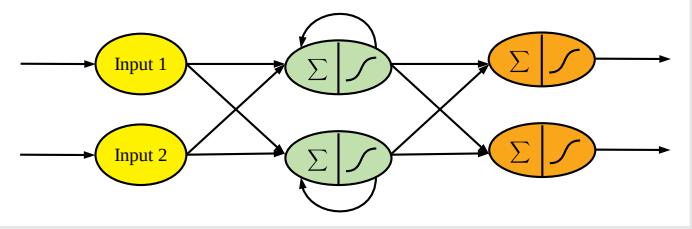
$$F_t = \sigma(W_{X_F} X_t + W_{Y_F} Y_{t-1} + B_F)$$

$$I_t = \sigma(W_{X_I} X_t + W_{Y_I} Y_{t-1} + B_I)$$



Deep learning for time series – LSTM

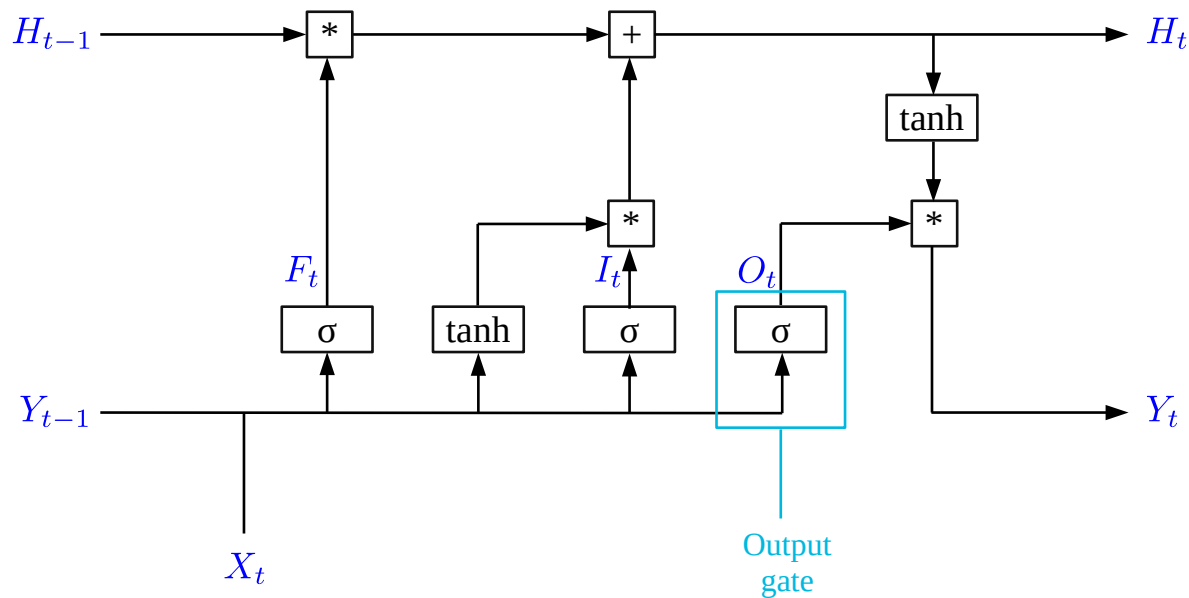
└ Anatomy of a LSTM (6/8)



$$F_t = \sigma(W_{X_F} X_t + W_{Y_F} Y_{t-1} + B_F)$$

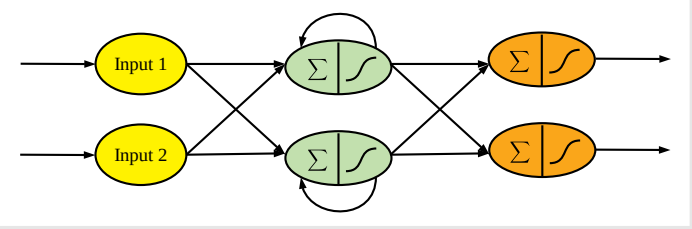
$$I_t = \sigma(W_{X_I} X_t + W_{Y_I} Y_{t-1} + B_I)$$

$$O_t = \sigma(W_{X_O} X_t + W_{Y_O} Y_{t-1} + B_O)$$



Deep learning for time series – LSTM

└ Anatomy of a LSTM (7/8)

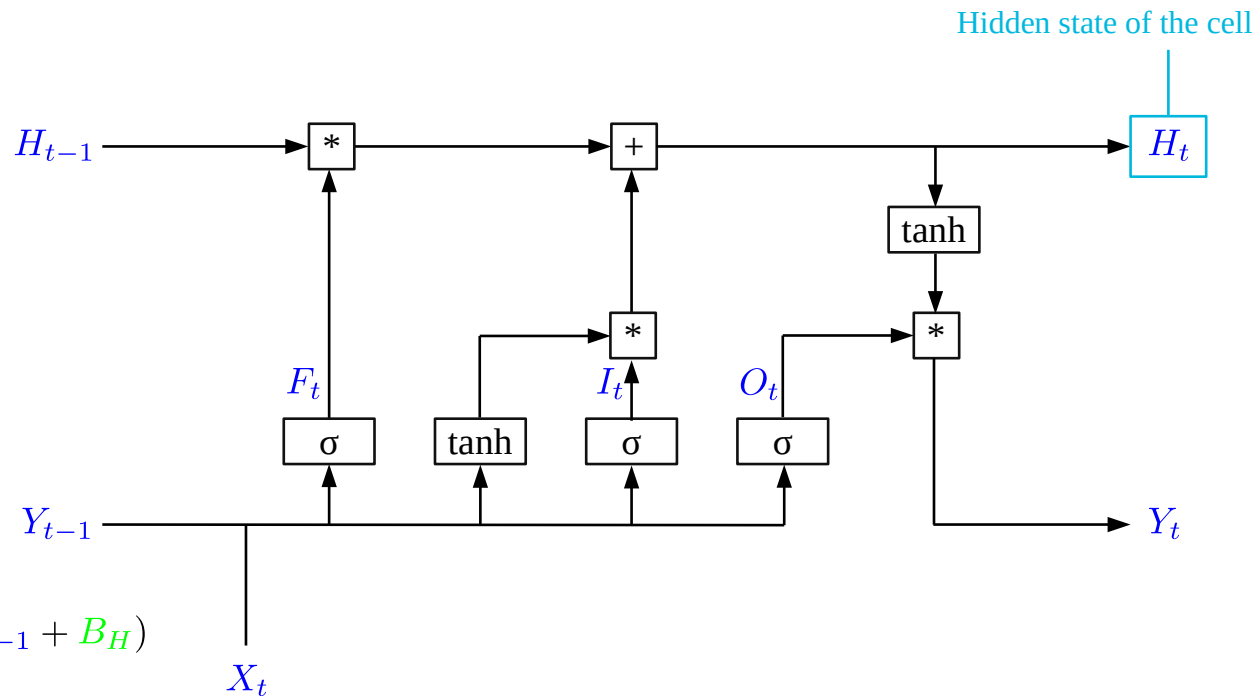


$$F_t = \sigma(W_{X_F} X_t + W_{Y_F} Y_{t-1} + B_F)$$

$$I_t = \sigma(W_{X_I} X_t + W_{Y_I} Y_{t-1} + B_I)$$

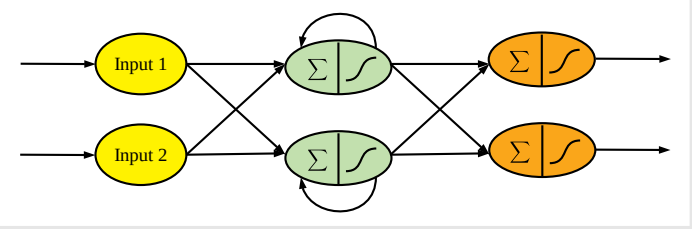
$$O_t = \sigma(W_{X_O} X_t + W_{Y_O} Y_{t-1} + B_O)$$

$$H_t = F_t \otimes H_{t-1} + I_t \otimes \tanh(W_{X_H} X_t + W_{Y_H} Y_{t-1} + B_H)$$



Deep learning for time series – LSTM

└ Anatomy of a LSTM (8/8)



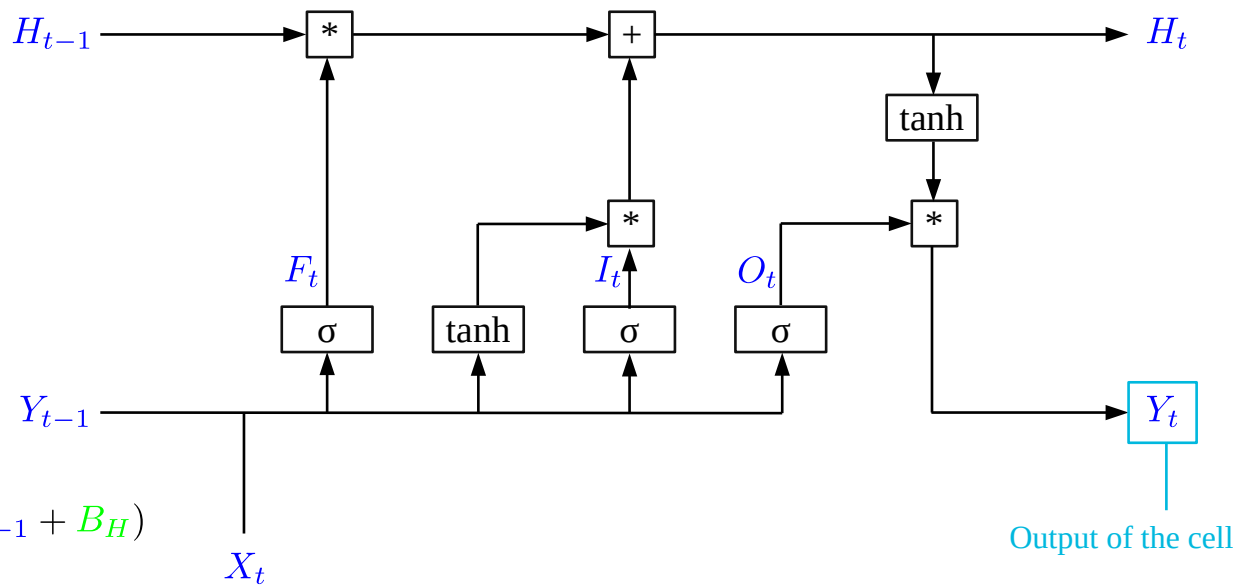
$$F_t = \sigma(W_{X_F} X_t + W_{Y_F} Y_{t-1} + B_F)$$

$$I_t = \sigma(W_{X_I} X_t + W_{Y_I} Y_{t-1} + B_I)$$

$$O_t = \sigma(W_{X_O} X_t + W_{Y_O} Y_{t-1} + B_O)$$

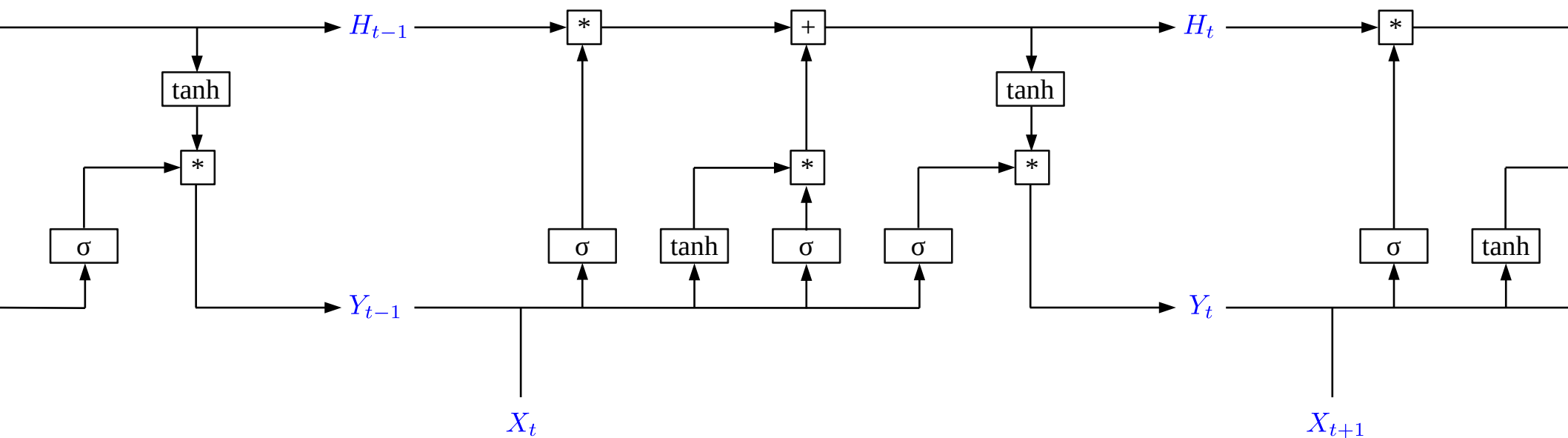
$$H_t = F_t \otimes H_{t-1} + I_t \otimes \tanh(W_{X_H} X_t + W_{Y_H} Y_{t-1} + B_H)$$

$$Y_t = O_t \otimes \tanh(H_t)$$



Deep learning for time series – LSTM

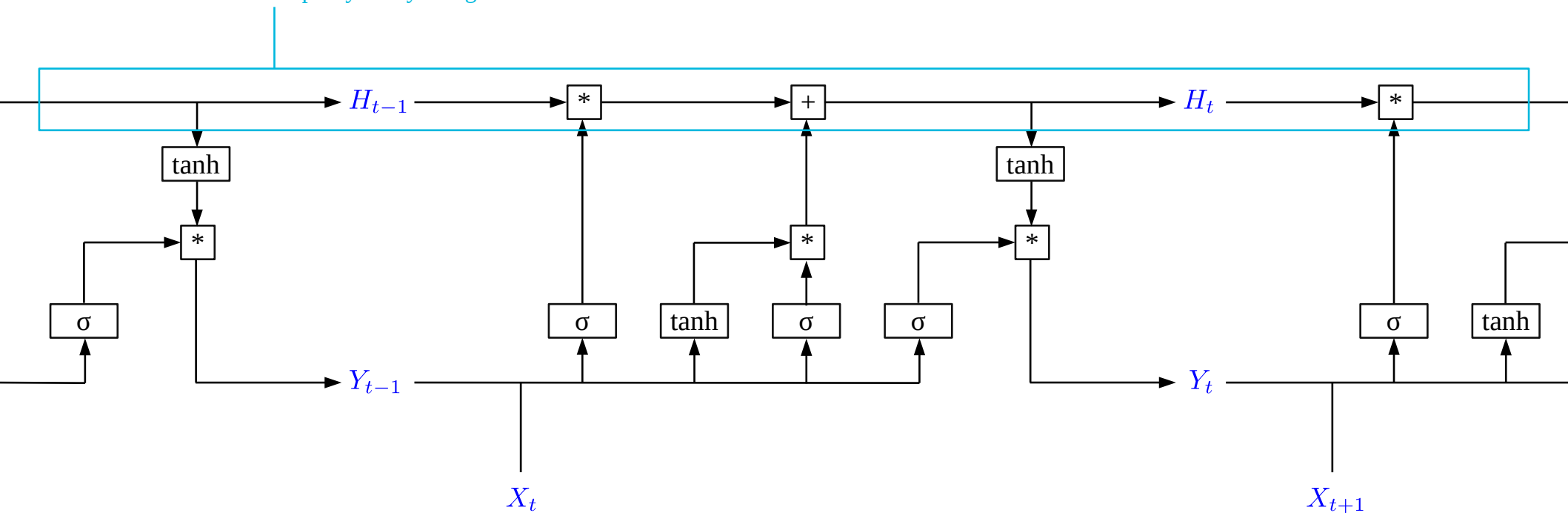
└ How does it work? (1/5)



Deep learning for time series – LSTM

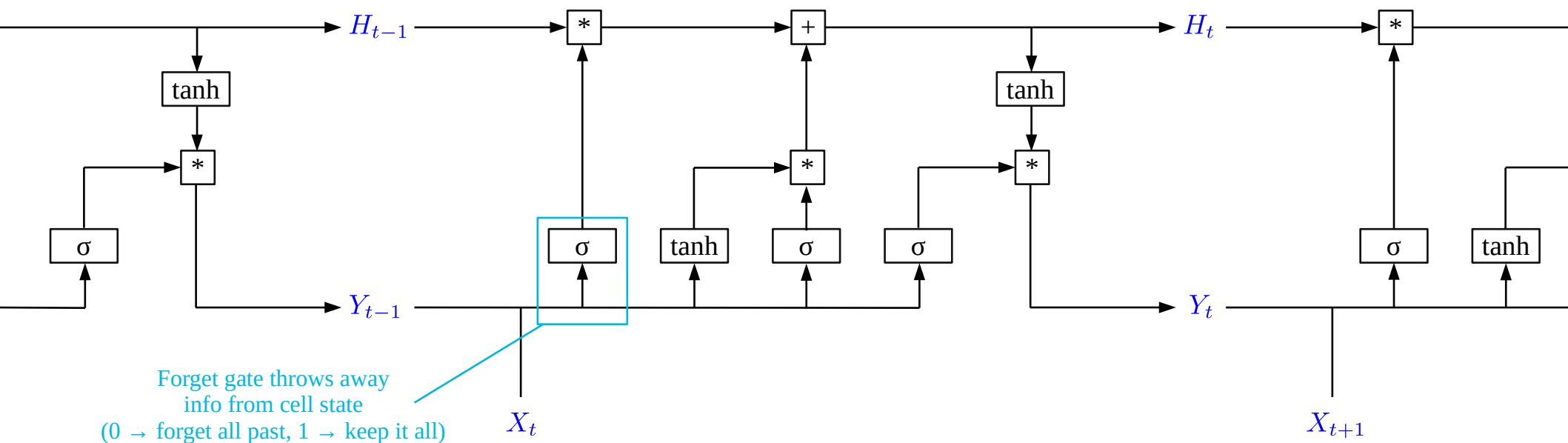
How does it work? (2/5)

Core idea: information flows pretty easily along this line



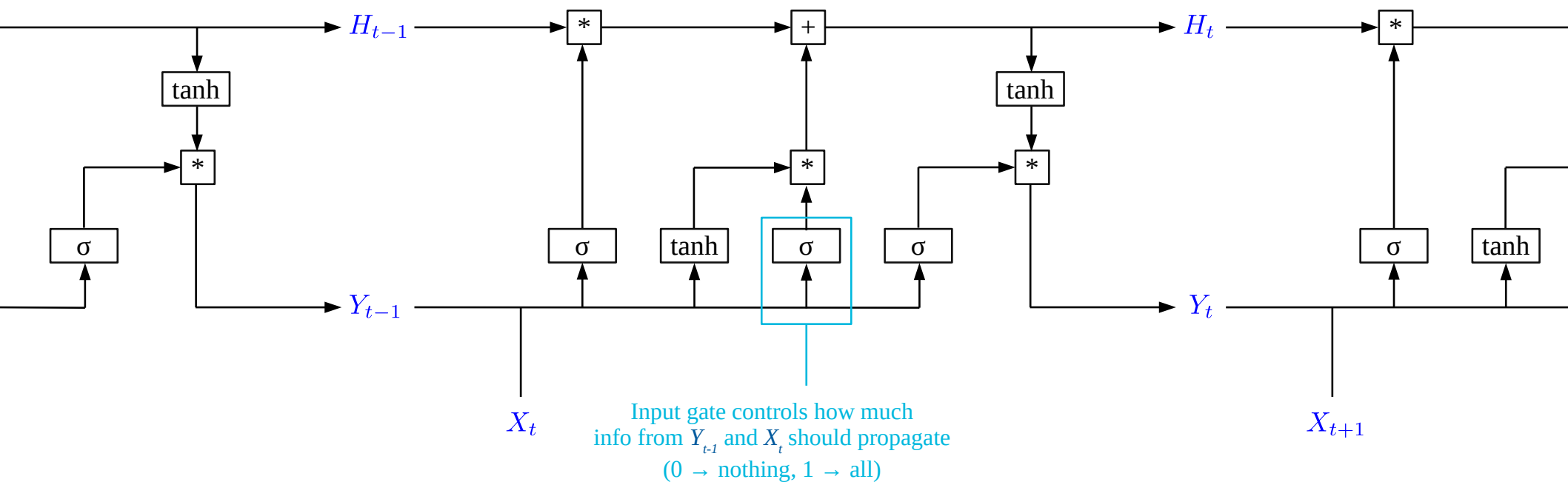
Deep learning for time series – LSTM

└ How does it work? (3/5)



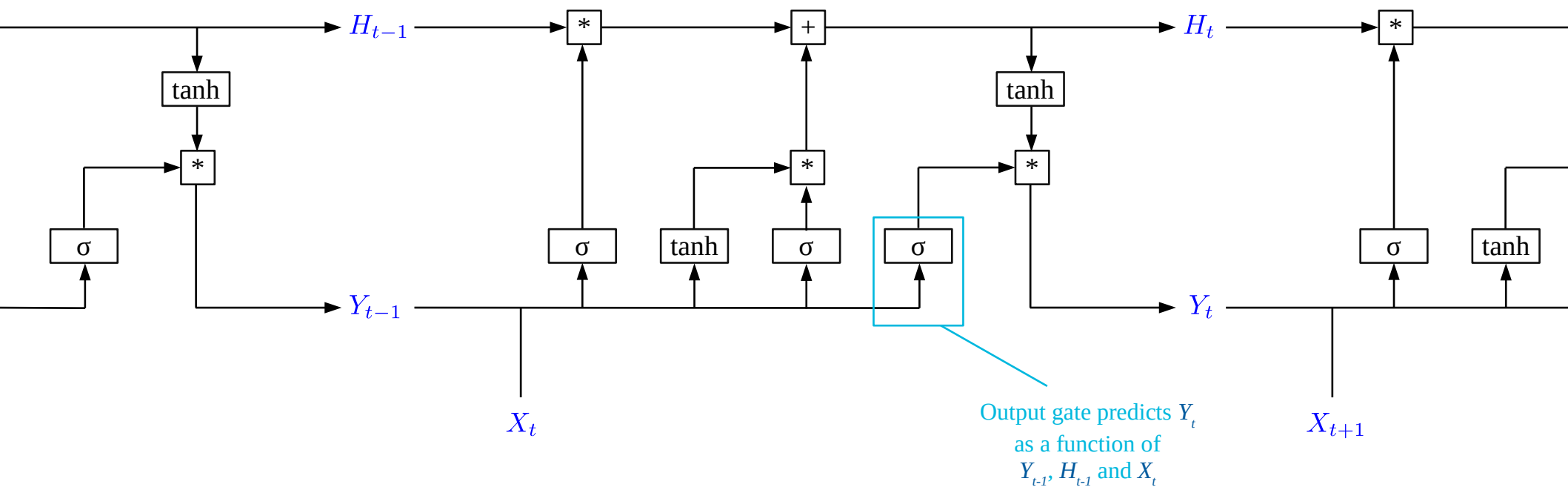
Deep learning for time series – LSTM

└ How does it work? (4/5)



Deep learning for time series – LSTM

└ How does it work? (5/5)



Deep learning for time series – LSTM

└ Gradient vanishing problem solved! (1/8)

$$\blacktriangleright \frac{\partial E_{t+T}}{\partial \mathbf{W}} = \frac{\partial E_{t+T}}{\partial \mathbf{Y}_{t+T}} \frac{\partial \mathbf{Y}_{t+T}}{\partial \mathbf{H}_{t+T}} \frac{\partial \mathbf{H}_{t+T}}{\partial \mathbf{H}_{t+T-1}} \frac{\partial \mathbf{H}_{t+T-1}}{\partial \mathbf{H}_{t+T-2}} \cdots \frac{\partial \mathbf{H}_{t+1}}{\partial \mathbf{H}_t} \frac{\partial \mathbf{H}_t}{\partial \mathbf{W}} = \frac{\partial E_{t+T}}{\partial \mathbf{Y}_{t+T}} \frac{\partial \mathbf{Y}_{t+T}}{\partial \mathbf{H}_{t+T}} \frac{\partial \mathbf{H}_{t+T}}{\partial \mathbf{H}_{t+T-1}} \left(\prod_{k=t+1}^{t+T-1} \frac{\partial \mathbf{H}_k}{\partial \mathbf{H}_{k-1}} \right) \frac{\partial \mathbf{H}_t}{\partial \mathbf{W}}$$

Deep learning for time series – LSTM

└ Gradient vanishing problem solved! (2/8)

$$\blacktriangleright \frac{\partial E_{t+T}}{\partial \mathbf{W}} = \frac{\partial E_{t+T}}{\partial \mathbf{Y}_{t+T}} \frac{\partial \mathbf{Y}_{t+T}}{\partial \mathbf{H}_{t+T}} \frac{\partial \mathbf{H}_{t+T}}{\partial \mathbf{H}_{t+T-1}} \frac{\partial \mathbf{H}_{t+T-1}}{\partial \mathbf{H}_{t+T-2}} \cdots \frac{\partial \mathbf{H}_{t+1}}{\partial \mathbf{H}_t} \frac{\partial \mathbf{H}_t}{\partial \mathbf{W}} = \frac{\partial E_{t+T}}{\partial \mathbf{Y}_{t+T}} \frac{\partial \mathbf{Y}_{t+T}}{\partial \mathbf{H}_{t+T}} \frac{\partial \mathbf{H}_{t+T}}{\partial \mathbf{H}_{t+T-1}} \left(\prod_{k=t+1}^{t+T-1} \frac{\partial \mathbf{H}_k}{\partial \mathbf{H}_{k-1}} \right) \frac{\partial \mathbf{H}_t}{\partial \mathbf{W}}$$

$$\begin{aligned} \blacktriangleright \text{Cell state can be written as: } \mathbf{H}_k &= \mathbf{F}_k \otimes \mathbf{H}_{k-1} + \mathbf{I}_k \otimes \tanh(\mathbf{W}_{X_H} \mathbf{X}_k + \mathbf{W}_{Y_H} \mathbf{Y}_{k-1} + \mathbf{B}_H) \\ &= \sigma(\mathbf{W}_{X_F} \mathbf{X}_k + \mathbf{W}_{Y_F} \mathbf{Y}_{k-1} + \mathbf{B}_F) \otimes \mathbf{H}_{k-1} + \mathbf{I}_k \otimes \tanh(\mathbf{W}_{X_H} \mathbf{X}_k + \mathbf{W}_{Y_H} \mathbf{Y}_{k-1} + \mathbf{B}_H) \end{aligned}$$

Deep learning for time series – LSTM

└ Gradient vanishing problem solved! (3/8)

$$\blacktriangleright \frac{\partial E_{t+T}}{\partial \mathbf{W}} = \frac{\partial E_{t+T}}{\partial \mathbf{Y}_{t+T}} \frac{\partial \mathbf{Y}_{t+T}}{\partial \mathbf{H}_{t+T}} \frac{\partial \mathbf{H}_{t+T}}{\partial \mathbf{H}_{t+T-1}} \frac{\partial \mathbf{H}_{t+T-1}}{\partial \mathbf{H}_{t+T-2}} \cdots \frac{\partial \mathbf{H}_{t+1}}{\partial \mathbf{H}_t} \frac{\partial \mathbf{H}_t}{\partial \mathbf{W}} = \frac{\partial E_{t+T}}{\partial \mathbf{Y}_{t+T}} \frac{\partial \mathbf{Y}_{t+T}}{\partial \mathbf{H}_{t+T}} \frac{\partial \mathbf{H}_{t+T}}{\partial \mathbf{H}_{t+T-1}} \left(\prod_{k=t+1}^{t+T-1} \frac{\partial \mathbf{H}_k}{\partial \mathbf{H}_{k-1}} \right) \frac{\partial \mathbf{H}_t}{\partial \mathbf{W}}$$

$$\begin{aligned} \blacktriangleright \text{Cell state can be written as: } \mathbf{H}_k &= \mathbf{F}_k \otimes \mathbf{H}_{k-1} + \mathbf{I}_k \otimes \tanh(\mathbf{W}_{X_H} \mathbf{X}_k + \mathbf{W}_{Y_H} \mathbf{Y}_{k-1} + \mathbf{B}_H) \\ &= \sigma(\mathbf{W}_{X_F} \mathbf{X}_k + \mathbf{W}_{Y_F} \mathbf{Y}_{k-1} + \mathbf{B}_F) \otimes \mathbf{H}_{k-1} + \mathbf{I}_k \otimes \tanh(\mathbf{W}_{X_H} \mathbf{X}_k + \mathbf{W}_{Y_H} \mathbf{Y}_{k-1} + \mathbf{B}_H) \end{aligned}$$

$$\blacktriangleright \text{Derivating } \mathbf{H}_k, \text{ we get: } \frac{\partial \mathbf{H}_k}{\partial \mathbf{H}_{k-1}} = \sigma(\mathbf{W}_{X_F} \mathbf{X}_k + \mathbf{W}_{Y_F} \mathbf{Y}_{k-1} + \mathbf{B}_F) + \frac{d}{d\mathbf{H}_{k-1}} [\mathbf{I}_k \otimes \tanh(\mathbf{W}_{X_H} \mathbf{X}_k + \mathbf{W}_{Y_H} \mathbf{Y}_{k-1} + \mathbf{B}_H)]$$

Deep learning for time series – LSTM

└ Gradient vanishing problem solved! (4/8)

$$\blacktriangleright \frac{\partial E_{t+T}}{\partial \mathbf{W}} = \frac{\partial E_{t+T}}{\partial \mathbf{Y}_{t+T}} \frac{\partial \mathbf{Y}_{t+T}}{\partial \mathbf{H}_{t+T}} \frac{\partial \mathbf{H}_{t+T}}{\partial \mathbf{H}_{t+T-1}} \frac{\partial \mathbf{H}_{t+T-1}}{\partial \mathbf{H}_{t+T-2}} \cdots \frac{\partial \mathbf{H}_{t+1}}{\partial \mathbf{H}_t} \frac{\partial \mathbf{H}_t}{\partial \mathbf{W}} = \frac{\partial E_{t+T}}{\partial \mathbf{Y}_{t+T}} \frac{\partial \mathbf{Y}_{t+T}}{\partial \mathbf{H}_{t+T}} \frac{\partial \mathbf{H}_{t+T}}{\partial \mathbf{H}_{t+T-1}} \left(\prod_{k=t+1}^{t+T-1} \frac{\partial \mathbf{H}_k}{\partial \mathbf{H}_{k-1}} \right) \frac{\partial \mathbf{H}_t}{\partial \mathbf{W}}$$

$$\begin{aligned} \blacktriangleright \text{Cell state can be written as: } \mathbf{H}_k &= \mathbf{F}_k \otimes \mathbf{H}_{k-1} + \mathbf{I}_k \otimes \tanh(\mathbf{W}_{X_H} \mathbf{X}_k + \mathbf{W}_{Y_H} \mathbf{Y}_{k-1} + \mathbf{B}_H) \\ &= \sigma(\mathbf{W}_{X_F} \mathbf{X}_k + \mathbf{W}_{Y_F} \mathbf{Y}_{k-1} + \mathbf{B}_F) \otimes \mathbf{H}_{k-1} + \mathbf{I}_k \otimes \tanh(\mathbf{W}_{X_H} \mathbf{X}_k + \mathbf{W}_{Y_H} \mathbf{Y}_{k-1} + \mathbf{B}_H) \end{aligned}$$

$$\blacktriangleright \text{Derivating } \mathbf{H}_k, \text{ we get: } \frac{\partial \mathbf{H}_k}{\partial \mathbf{H}_{k-1}} = \sigma(\mathbf{W}_{X_F} \mathbf{X}_k + \mathbf{W}_{Y_F} \mathbf{Y}_{k-1} + \mathbf{B}_F) + \frac{d}{d\mathbf{H}_{k-1}} [\mathbf{I}_k \otimes \tanh(\mathbf{W}_{X_H} \mathbf{X}_k + \mathbf{W}_{Y_H} \mathbf{Y}_{k-1} + \mathbf{B}_H)]$$

Let's put this aside (of little interest for the demonstration)

Deep learning for time series – LSTM

└ Gradient vanishing problem solved! (5/8)

$$\blacktriangleright \frac{\partial E_{t+T}}{\partial \mathbf{W}} = \frac{\partial E_{t+T}}{\partial \mathbf{Y}_{t+T}} \frac{\partial \mathbf{Y}_{t+T}}{\partial \mathbf{H}_{t+T}} \frac{\partial \mathbf{H}_{t+T}}{\partial \mathbf{H}_{t+T-1}} \frac{\partial \mathbf{H}_{t+T-1}}{\partial \mathbf{H}_{t+T-2}} \cdots \frac{\partial \mathbf{H}_{t+1}}{\partial \mathbf{H}_t} \frac{\partial \mathbf{H}_t}{\partial \mathbf{W}} = \frac{\partial E_{t+T}}{\partial \mathbf{Y}_{t+T}} \frac{\partial \mathbf{Y}_{t+T}}{\partial \mathbf{H}_{t+T}} \frac{\partial \mathbf{H}_{t+T}}{\partial \mathbf{H}_{t+T-1}} \left(\prod_{k=t+1}^{t+T-1} \frac{\partial \mathbf{H}_k}{\partial \mathbf{H}_{k-1}} \right) \frac{\partial \mathbf{H}_t}{\partial \mathbf{W}}$$

$$\begin{aligned} \blacktriangleright \text{Cell state can be written as: } \mathbf{H}_k &= \mathbf{F}_k \otimes \mathbf{H}_{k-1} + \mathbf{I}_k \otimes \tanh(\mathbf{W}_{X_H} \mathbf{X}_k + \mathbf{W}_{Y_H} \mathbf{Y}_{k-1} + \mathbf{B}_H) \\ &= \sigma(\mathbf{W}_{X_F} \mathbf{X}_k + \mathbf{W}_{Y_F} \mathbf{Y}_{k-1} + \mathbf{B}_F) \otimes \mathbf{H}_{k-1} + \mathbf{I}_k \otimes \tanh(\mathbf{W}_{X_H} \mathbf{X}_k + \mathbf{W}_{Y_H} \mathbf{Y}_{k-1} + \mathbf{B}_H) \end{aligned}$$

$$\blacktriangleright \text{Derivating } \mathbf{H}_k, \text{ we get: } \frac{\partial \mathbf{H}_k}{\partial \mathbf{H}_{k-1}} = \sigma(\mathbf{W}_{X_F} \mathbf{X}_k + \mathbf{W}_{Y_F} \mathbf{Y}_{k-1} + \mathbf{B}_F) + \dots$$

Deep learning for time series – LSTM

└ Gradient vanishing problem solved! (6/8)

$$\blacktriangleright \frac{\partial E_{t+T}}{\partial \mathbf{W}} = \frac{\partial E_{t+T}}{\partial \mathbf{Y}_{t+T}} \frac{\partial \mathbf{Y}_{t+T}}{\partial \mathbf{H}_{t+T}} \frac{\partial \mathbf{H}_{t+T}}{\partial \mathbf{H}_{t+T-1}} \frac{\partial \mathbf{H}_{t+T-1}}{\partial \mathbf{H}_{t+T-2}} \cdots \frac{\partial \mathbf{H}_{t+1}}{\partial \mathbf{H}_t} \frac{\partial \mathbf{H}_t}{\partial \mathbf{W}} = \frac{\partial E_{t+T}}{\partial \mathbf{Y}_{t+T}} \frac{\partial \mathbf{Y}_{t+T}}{\partial \mathbf{H}_{t+T}} \frac{\partial \mathbf{H}_{t+T}}{\partial \mathbf{H}_{t+T-1}} \left(\prod_{k=t+1}^{t+T-1} \frac{\partial \mathbf{H}_k}{\partial \mathbf{H}_{k-1}} \right) \frac{\partial \mathbf{H}_t}{\partial \mathbf{W}}$$

$$\begin{aligned} \blacktriangleright \text{Cell state can be written as: } \mathbf{H}_k &= \mathbf{F}_k \otimes \mathbf{H}_{k-1} + \mathbf{I}_k \otimes \tanh(\mathbf{W}_{X_H} \mathbf{X}_k + \mathbf{W}_{Y_H} \mathbf{Y}_{k-1} + \mathbf{B}_H) \\ &= \sigma(\mathbf{W}_{X_F} \mathbf{X}_k + \mathbf{W}_{Y_F} \mathbf{Y}_{k-1} + \mathbf{B}_F) \otimes \mathbf{H}_{k-1} + \mathbf{I}_k \otimes \tanh(\mathbf{W}_{X_H} \mathbf{X}_k + \mathbf{W}_{Y_H} \mathbf{Y}_{k-1} + \mathbf{B}_H) \end{aligned}$$

$$\blacktriangleright \text{Derivating } \mathbf{H}_k, \text{ we get: } \frac{\partial \mathbf{H}_k}{\partial \mathbf{H}_{k-1}} = \sigma(\mathbf{W}_{X_F} \mathbf{X}_k + \mathbf{W}_{Y_F} \mathbf{Y}_{k-1} + \mathbf{B}_F) + \dots$$

► First conclusion: the forget gate controls gradient behavior

Deep learning for time series – LSTM

└ Gradient vanishing problem solved! (7/8)

$$\blacktriangleright \frac{\partial E_{t+T}}{\partial \mathbf{W}} = \frac{\partial E_{t+T}}{\partial \mathbf{Y}_{t+T}} \frac{\partial \mathbf{Y}_{t+T}}{\partial \mathbf{H}_{t+T}} \frac{\partial \mathbf{H}_{t+T}}{\partial \mathbf{H}_{t+T-1}} \frac{\partial \mathbf{H}_{t+T-1}}{\partial \mathbf{H}_{t+T-2}} \cdots \frac{\partial \mathbf{H}_{t+1}}{\partial \mathbf{H}_t} \frac{\partial \mathbf{H}_t}{\partial \mathbf{W}} = \frac{\partial E_{t+T}}{\partial \mathbf{Y}_{t+T}} \frac{\partial \mathbf{Y}_{t+T}}{\partial \mathbf{H}_{t+T}} \frac{\partial \mathbf{H}_{t+T}}{\partial \mathbf{H}_{t+T-1}} \left(\prod_{k=t+1}^{t+T-1} \frac{\partial \mathbf{H}_k}{\partial \mathbf{H}_{k-1}} \right) \frac{\partial \mathbf{H}_t}{\partial \mathbf{W}}$$

$$\begin{aligned} \blacktriangleright \text{Cell state can be written as: } \mathbf{H}_k &= \mathbf{F}_k \otimes \mathbf{H}_{k-1} + \mathbf{I}_k \otimes \tanh(\mathbf{W}_{X_H} \mathbf{X}_k + \mathbf{W}_{Y_H} \mathbf{Y}_{k-1} + \mathbf{B}_H) \\ &= \sigma(\mathbf{W}_{X_F} \mathbf{X}_k + \mathbf{W}_{Y_F} \mathbf{Y}_{k-1} + \mathbf{B}_F) \otimes \mathbf{H}_{k-1} + \mathbf{I}_k \otimes \tanh(\mathbf{W}_{X_H} \mathbf{X}_k + \mathbf{W}_{Y_H} \mathbf{Y}_{k-1} + \mathbf{B}_H) \end{aligned}$$

$$\blacktriangleright \text{Derivating } \mathbf{H}_k, \text{ we get: } \frac{\partial \mathbf{H}_k}{\partial \mathbf{H}_{k-1}} = \sigma(\mathbf{W}_{X_F} \mathbf{X}_k + \mathbf{W}_{Y_F} \mathbf{Y}_{k-1} + \mathbf{B}_F) + \dots$$

► First conclusion: the forget gate controls gradient behavior

$$\blacktriangleright \text{For simplicity, let's think of forget gate as } \sigma(\mathbf{W}_{X_F} \mathbf{X}_k + \mathbf{W}_{Y_F} \mathbf{Y}_{k-1} + \mathbf{B}_F) \approx \vec{1}, \text{ thus: } \frac{\partial \mathbf{H}_k}{\partial \mathbf{H}_{k-1}} \not\rightarrow 0$$

Deep learning for time series – LSTM

└ Gradient vanishing problem solved! (8/8)

$$\triangleright \frac{\partial E_{t+T}}{\partial \mathbf{W}} = \frac{\partial E_{t+T}}{\partial \mathbf{Y}_{t+T}} \frac{\partial \mathbf{Y}_{t+T}}{\partial \mathbf{H}_{t+T}} \frac{\partial \mathbf{H}_{t+T}}{\partial \mathbf{H}_{t+T-1}} \frac{\partial \mathbf{H}_{t+T-1}}{\partial \mathbf{H}_{t+T-2}} \cdots \frac{\partial \mathbf{H}_{t+1}}{\partial \mathbf{H}_t} \frac{\partial \mathbf{H}_t}{\partial \mathbf{W}} = \frac{\partial E_{t+T}}{\partial \mathbf{Y}_{t+T}} \frac{\partial \mathbf{Y}_{t+T}}{\partial \mathbf{H}_{t+T}} \frac{\partial \mathbf{H}_{t+T}}{\partial \mathbf{H}_{t+T-1}} \left(\prod_{k=t+1}^{t+T-1} \frac{\partial \mathbf{H}_k}{\partial \mathbf{H}_{k-1}} \right) \frac{\partial \mathbf{H}_t}{\partial \mathbf{W}}$$

$$\begin{aligned} \triangleright \text{Cell state can be written as: } \mathbf{H}_k &= \mathbf{F}_k \otimes \mathbf{H}_{k-1} + \mathbf{I}_k \otimes \tanh(\mathbf{W}_{X_H} \mathbf{X}_k + \mathbf{W}_{Y_H} \mathbf{Y}_{k-1} + \mathbf{B}_H) \\ &= \sigma(\mathbf{W}_{X_F} \mathbf{X}_k + \mathbf{W}_{Y_F} \mathbf{Y}_{k-1} + \mathbf{B}_F) \otimes \mathbf{H}_{k-1} + \mathbf{I}_k \otimes \tanh(\mathbf{W}_{X_H} \mathbf{X}_k + \mathbf{W}_{Y_H} \mathbf{Y}_{k-1} + \mathbf{B}_H) \end{aligned}$$

$$\triangleright \text{Derivating } \mathbf{H}_k, \text{ we get: } \frac{\partial \mathbf{H}_k}{\partial \mathbf{H}_{k-1}} = \sigma(\mathbf{W}_{X_F} \mathbf{X}_k + \mathbf{W}_{Y_F} \mathbf{Y}_{k-1} + \mathbf{B}_F) + \dots$$

► First conclusion: the forget gate controls gradient behavior

$$\triangleright \text{For simplicity, let's think of forget gate as } \sigma(\mathbf{W}_{X_F} \mathbf{X}_k + \mathbf{W}_{Y_F} \mathbf{Y}_{k-1} + \mathbf{B}_F) \approx \vec{1}, \text{ thus: } \frac{\partial \mathbf{H}_k}{\partial \mathbf{H}_{k-1}} \not\rightarrow 0$$

$$\triangleright \text{Second conclusion: } \frac{\partial E_{t+T}}{\partial \mathbf{W}} \not\rightarrow 0, \text{ and gradient does not vanish anymore!}$$

Deep learning for time series – LSTM

└ A few references (1/2)

► Some LSTM extensions:

-  F. A. Gers & J. Schmidhuber – *Recurrent Nets that Time and Count* (2000)
<http://ftp.idsia.ch/pub/juergen/TimeCount-IJCNN2000.pdf> Peephole connections to let input and forget gates use H_{t-1}
-  K. Cho *et al.* – *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation* (2014)
<https://arxiv.org/pdf/1406.1078v3.pdf> Gated Recurrent Unit (GRU)
-  K. Yao *et al.* – *Depth-Gated Recurrent Neural Networks* (2015)
<https://arxiv.org/pdf/1508.03790v2.pdf> No explicit cell state
-  N. Kalchbrenner *et al.* – *Grid Long-Short Term Memory* (2015)
<https://arxiv.org/pdf/1507.01526v1.pdf> Spatiotemporal variant

► An example alternative to LSTMs:

-  J. Koutník *et al.* – *A Clockwork RNN* (2014)
<https://arxiv.org/pdf/1402.3511v1.pdf> Alternative to LSTMs for modeling long-term dependencies

► Reviews and more complete studies:

-  K. Greff *et al.* – *LSTM: A Search Space Odyssey* (2017)
<https://arxiv.org/pdf/1503.04069.pdf> A review paper on LSTMs and their variants
-  R. Jozefowicz *et al.* – *An Empirical Exploration of Recurrent Network Architectures* (2017)
<http://proceedings.mlr.press/v37/jozefowicz15.pdf> Comparison of 10k+ RNNs & LSTMs

Deep learning for time series – LSTM

└ A few references (2/2)

► A few applications:

-  K. Gregor *et al.* – *DRAW: A Recurrent Neural Network For Image Generation* (2000)
<https://arxiv.org/pdf/1502.04623.pdf> Generative models
-  H. Mayer *et al.* – *A System for Robotic Heart Surgery that Learns to Tie Knots Using Recurrent Neural Networks* (2006)
<ftp://ftp.idsia.ch/pub/juergen/iros06.pdf> Robotics
-  D. Eck & J. Schmidhuber – *Learning The Long-Term Structure of the Blues?* (2002)
ftp://ftp.idsia.ch/pub/juergen/2002_icannMusic.pdf Music generation
-  M. Baccouche – *Sequential Deep Learning for Human Action Recognition* (2011)
<https://liris.cnrs.fr/Documents/Liris-5228.pdf> Action recognition

► Cool online resources:

- @ C. Olah – *Understanding LSTM Networks* (2015)
<http://colah.github.io/posts/2015-08-Understanding-LSTMs/> Great blog on neural networks in general
- @ P. Srivastava – *Essentials of Deep Learning : Introduction to Long Short Term Memory* (2017)
<https://www.analyticsvidhya.com/blog/2017/12/fundamentals-of-deep-learning-introduction-to-lstm/> Code for texte generation using LSTMs
- @ P. Srivastava – *Recurrent Neural Networks (RNN) and Long Short-Term Memory (LSTM)* (2017)
<https://www.youtube.com/watch?v=WCUNPb-5EYI> A clear Youtube video
- @ A. Smola – *Gated Recurrent Unit (GRU)* (2019)
<https://www.youtube.com/watch?v=8Q582ng8Lxo> Berkeley course on GRUs in video

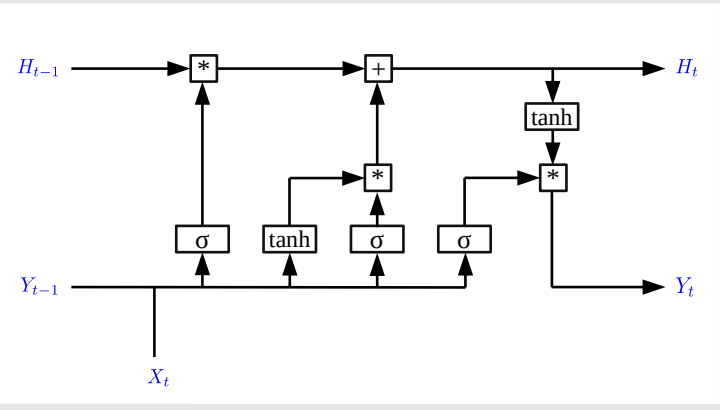
OUTLINE

- [Reminder] Neural networks 101
- Deep learning for time series:
 - Recurrent neural network (RNN)
 - Long short-term memory (LSTM)
 - Gated recurrent unit (GRU)
 - 1D convolutional neural network (1D-CNN)
- Transformers



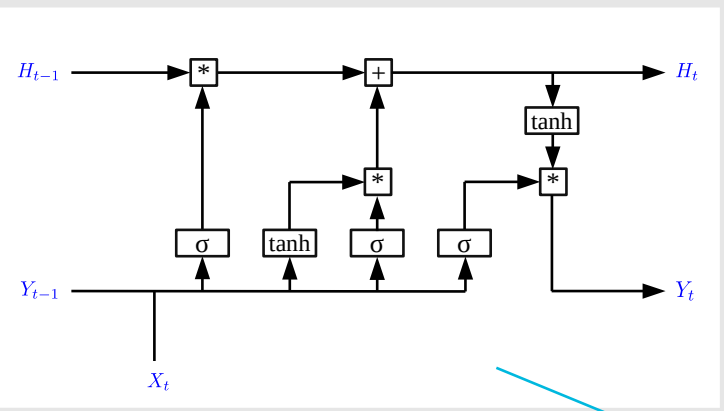
Deep learning for time series – GRU

└ Anatomy of a GRU (1/6)

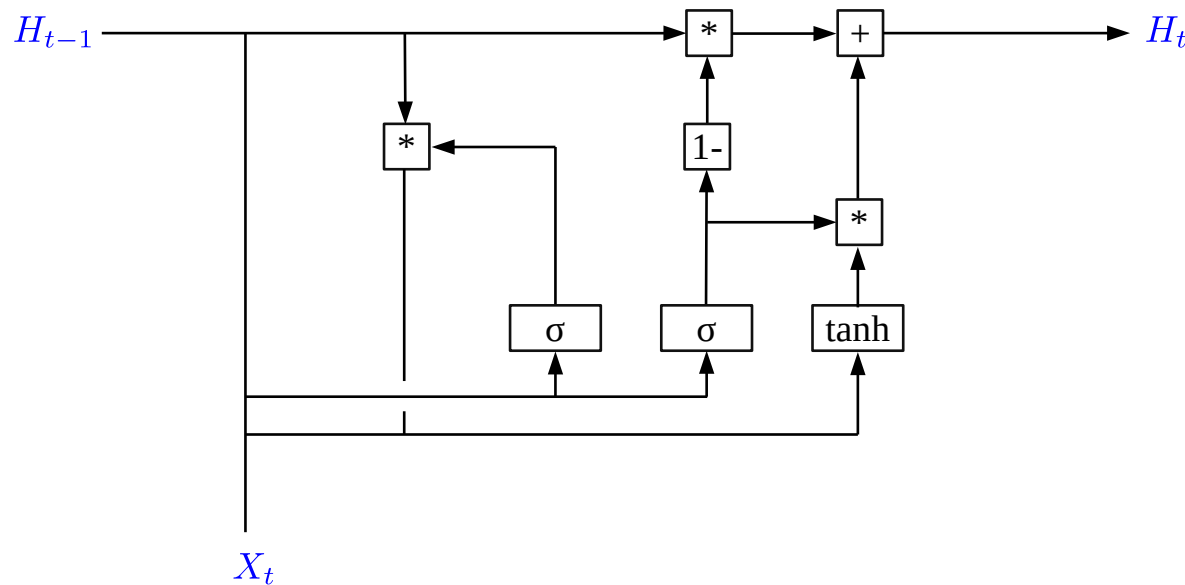


Deep learning for time series – GRU

└ Anatomy of a GRU (2/6)

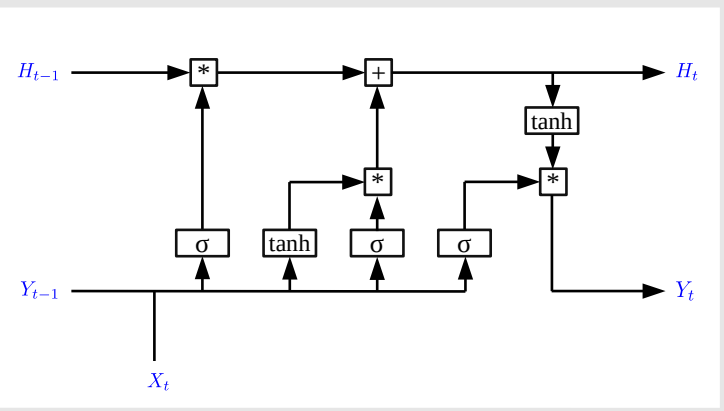


Replace with something
more simple

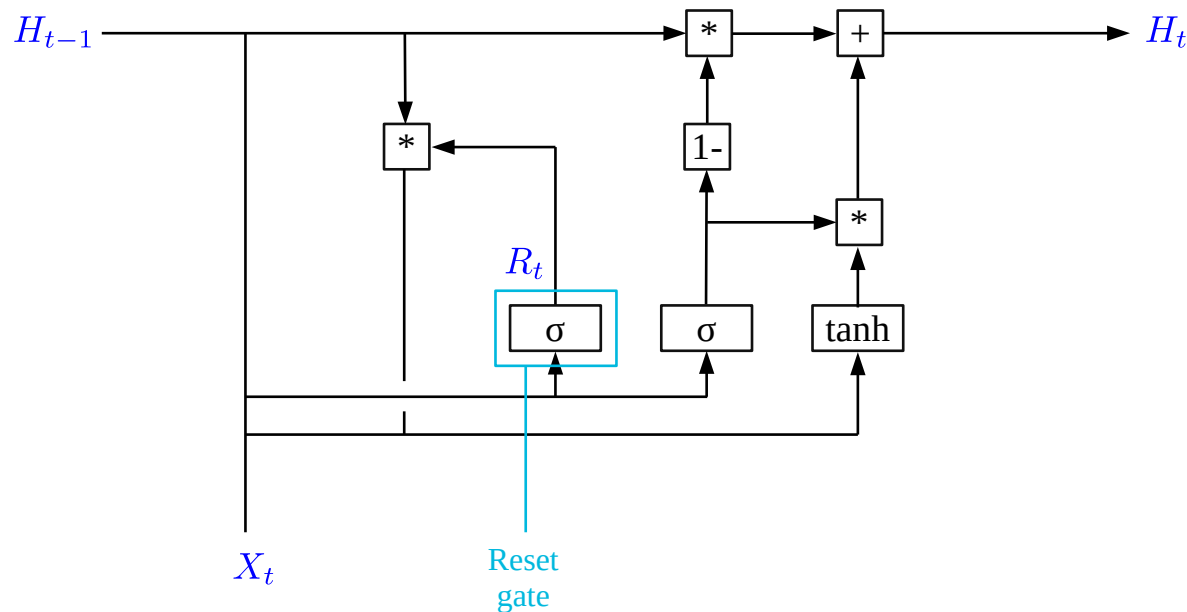


Deep learning for time series – GRU

└ Anatomy of a GRU (3/6)

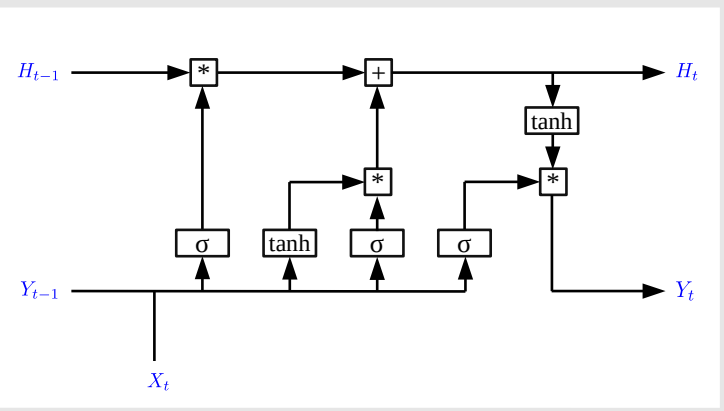


$$R_t = \sigma(W_{X_R} X_t + W_{H_R} H_{t-1} + B_R)$$



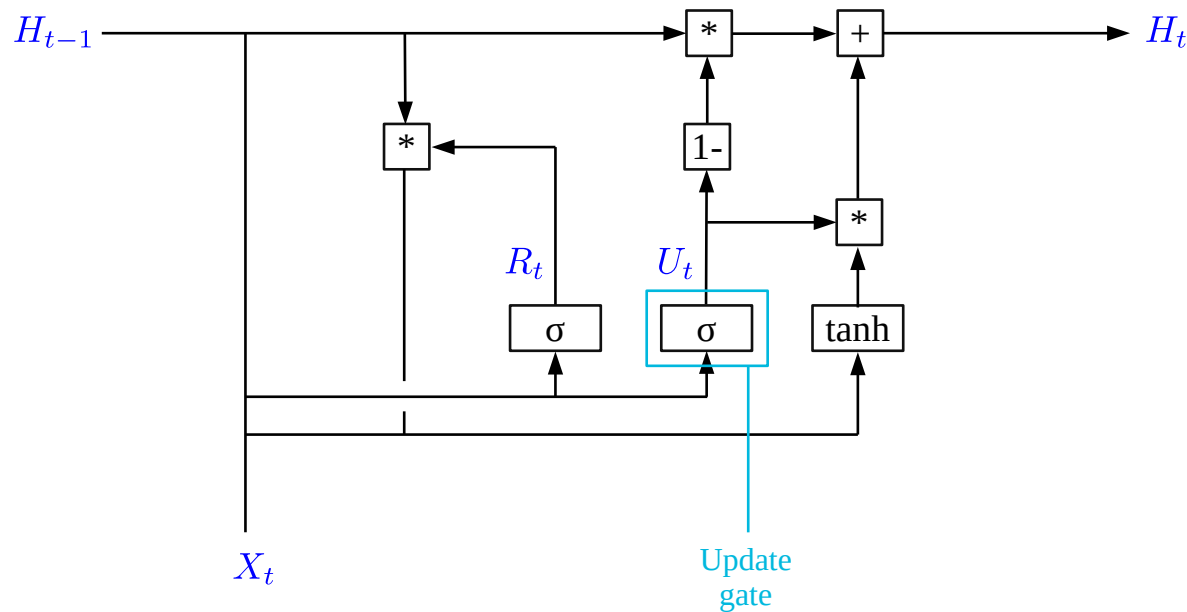
Deep learning for time series – GRU

└ Anatomy of a GRU (4/6)



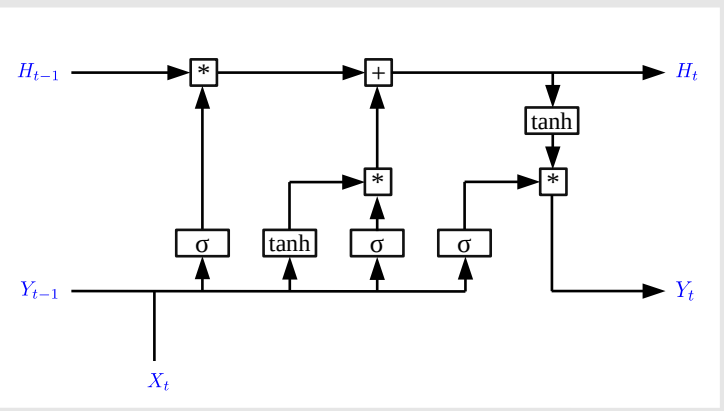
$$R_t = \sigma(W_{X_R}X_t + W_{H_R}H_{t-1} + B_R)$$

$$U_t = \sigma(W_{X_U}X_t + W_{H_U}H_{t-1} + B_U)$$



Deep learning for time series – GRU

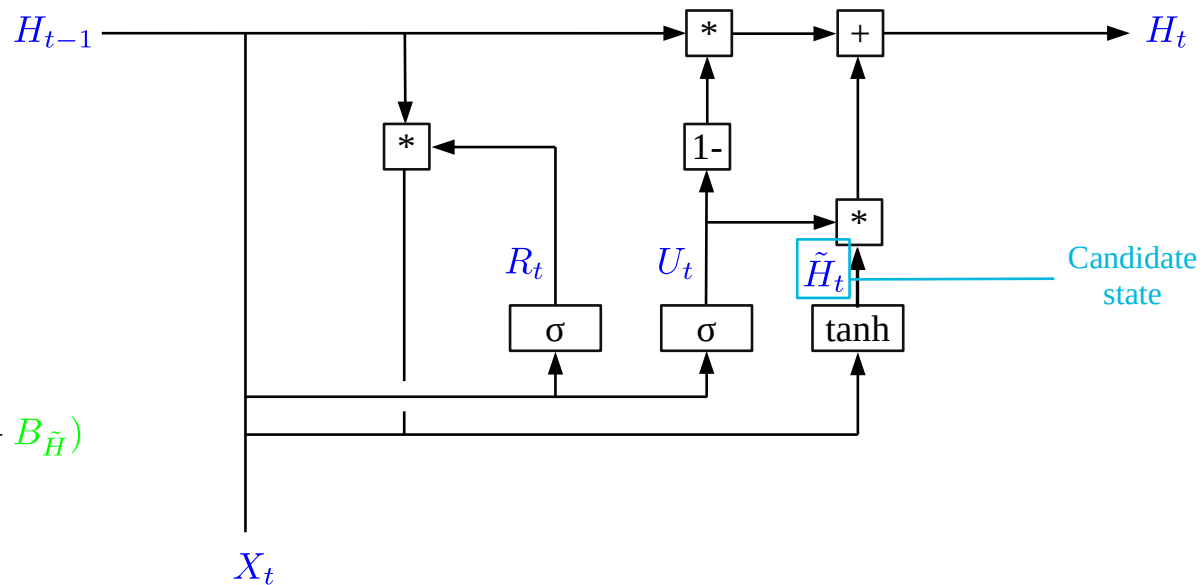
└ Anatomy of a GRU (5/6)



$$R_t = \sigma(W_{X_R}X_t + W_{H_R}H_{t-1} + B_R)$$

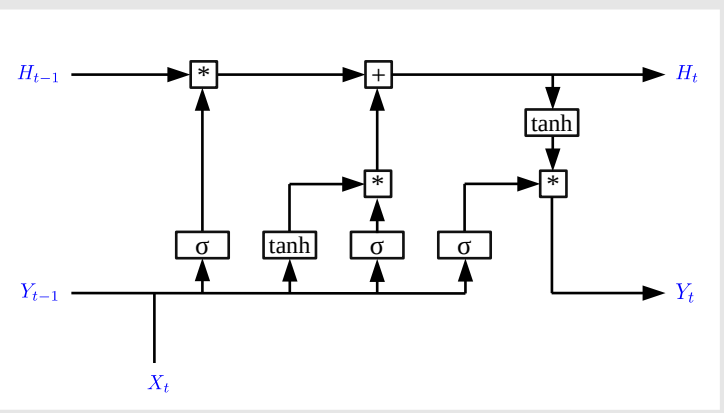
$$U_t = \sigma(W_{X_U}X_t + W_{H_U}H_{t-1} + B_U)$$

$$\tilde{H}_t = \tanh(W_{X_{\tilde{H}}}X_t + W_{H_{\tilde{H}}}(R_t \otimes H_{t-1}) + B_{\tilde{H}})$$



Deep learning for time series – GRU

└ Anatomy of a GRU (6/6)

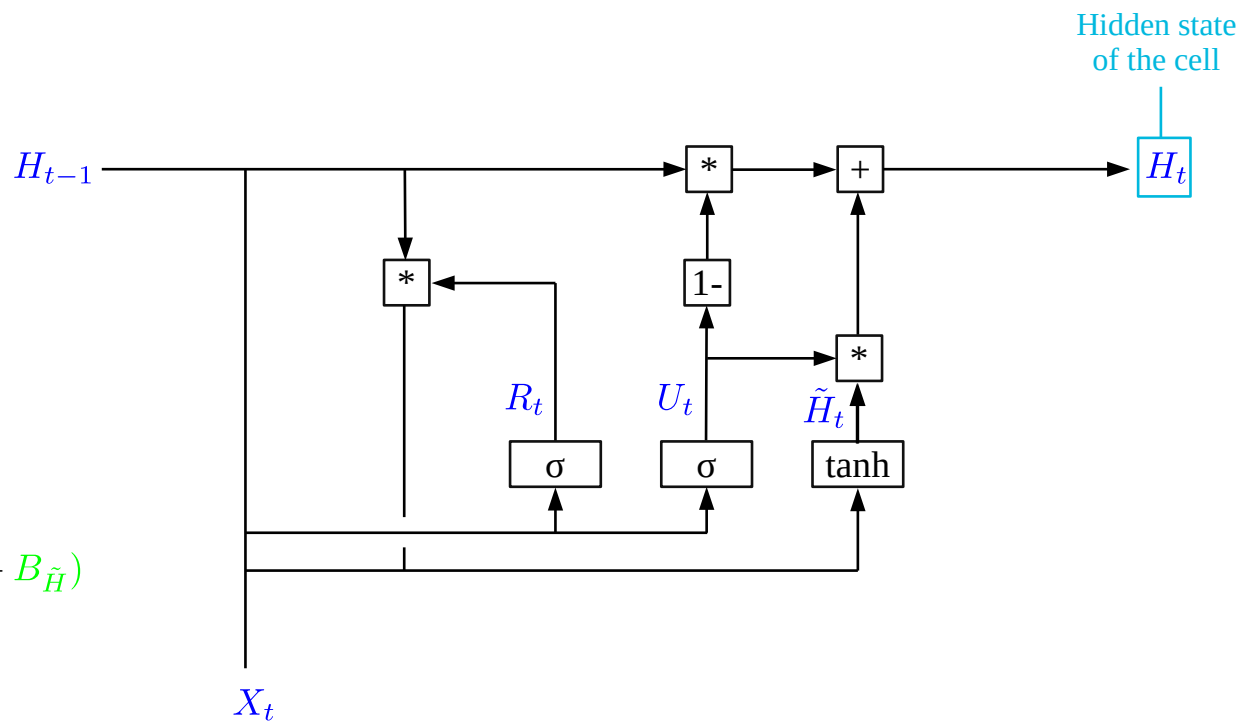


$$R_t = \sigma(W_{X_R}X_t + W_{H_R}H_{t-1} + B_R)$$

$$U_t = \sigma(W_{X_U}X_t + W_{H_U}H_{t-1} + B_U)$$

$$\tilde{H}_t = \tanh(W_{X_{\tilde{H}}}X_t + W_{H_{\tilde{H}}}(R_t \otimes H_{t-1}) + B_{\tilde{H}})$$

$$H_t = (1 - U_t) \otimes H_{t-1} + U_t \otimes \tilde{H}_t$$



Deep learning for time series – GRU

└ GRU vs. LSTM (1/4)

<https://arxiv.org/pdf/1412.3555v1.pdf>

GRU

$$R_t = \sigma(W_{X_R} X_t + W_{H_R} H_{t-1} + B_R)$$

$$U_t = \sigma(W_{X_U} X_t + W_{H_U} H_{t-1} + B_U)$$

$$\tilde{H}_t = \tanh(W_{X_{\tilde{H}}} X_t + W_{H_{\tilde{H}}} (R_t \otimes H_{t-1}) + B_{\tilde{H}})$$

$$H_t = (1 - U_t) \otimes H_{t-1} + U_t \otimes \tilde{H}_t$$

LSTM

$$F_t = \sigma(W_{X_F} X_t + W_{Y_F} Y_{t-1} + B_F)$$

$$I_t = \sigma(W_{X_I} X_t + W_{Y_I} Y_{t-1} + B_I)$$

$$O_t = \sigma(W_{X_O} X_t + W_{Y_O} Y_{t-1} + B_O)$$

$$H_t = F_t \otimes H_{t-1} + I_t \otimes \tanh(W_{X_H} X_t + W_{Y_H} Y_{t-1} + B_H)$$

$$Y_t = O_t \otimes \tanh(H_t)$$

Deep learning for time series – GRU

└ GRU vs. LSTM (2/4)

<https://arxiv.org/pdf/1412.3555v1.pdf>

GRU

$$R_t = \sigma(W_{X_R} X_t + W_{H_R} H_{t-1} + B_R)$$

$$U_t = \sigma(W_{X_U} X_t + W_{H_U} H_{t-1} + B_U)$$

$$\tilde{H}_t = \tanh(W_{X_{\tilde{H}}} X_t + W_{H_{\tilde{H}}} (R_t \otimes H_{t-1}) + B_{\tilde{H}})$$

$$H_t = (1 - U_t) \otimes H_{t-1} + U_t \otimes \tilde{H}_t$$

Simpler in design and in training

LSTM

$$F_t = \sigma(W_{X_F} X_t + W_{Y_F} Y_{t-1} + B_F)$$

$$I_t = \sigma(W_{X_I} X_t + W_{Y_I} Y_{t-1} + B_I)$$

$$O_t = \sigma(W_{X_O} X_t + W_{Y_O} Y_{t-1} + B_O)$$

$$H_t = F_t \otimes H_{t-1} + I_t \otimes \tanh(W_{X_H} X_t + W_{Y_H} Y_{t-1} + B_H)$$

$$Y_t = O_t \otimes \tanh(H_t)$$

More weights to train

Deep learning for time series – GRU

└ GRU vs. LSTM (3/4)

<https://arxiv.org/pdf/1412.3555v1.pdf>

GRU

$$R_t = \sigma(W_{X_R} X_t + W_{H_R} H_{t-1} + B_R)$$

$$U_t = \sigma(W_{X_U} X_t + W_{H_U} H_{t-1} + B_U)$$

$$\tilde{H}_t = \tanh(W_{X_{\tilde{H}}} X_t + W_{H_{\tilde{H}}} (R_t \otimes H_{t-1}) + B_{\tilde{H}})$$

$$H_t = (1 - U_t) \otimes H_{t-1} + U_t \otimes \tilde{H}_t$$

Simpler in design and in training

Seems to be more efficient in general, especially with less training data

LSTM

$$F_t = \sigma(W_{X_F} X_t + W_{Y_F} Y_{t-1} + B_F)$$

$$I_t = \sigma(W_{X_I} X_t + W_{Y_I} Y_{t-1} + B_I)$$

$$O_t = \sigma(W_{X_O} X_t + W_{Y_O} Y_{t-1} + B_O)$$

$$H_t = F_t \otimes H_{t-1} + I_t \otimes \tanh(W_{X_H} X_t + W_{Y_H} Y_{t-1} + B_H)$$

$$Y_t = O_t \otimes \tanh(H_t)$$

More weights to train

Seems to outperform GRU for applications with very long dependencies

Deep learning for time series – GRU

└ GRU vs. LSTM (4/4)

<https://arxiv.org/pdf/1412.3555v1.pdf>

GRU

$$R_t = \sigma(W_{X_R} X_t + W_{H_R} H_{t-1} + B_R)$$

$$U_t = \sigma(W_{X_U} X_t + W_{H_U} H_{t-1} + B_U)$$

$$\tilde{H}_t = \tanh(W_{X_{\tilde{H}}} X_t + W_{H_{\tilde{H}}} (R_t \otimes H_{t-1}) + B_{\tilde{H}})$$

$$H_t = (1 - U_t) \otimes H_{t-1} + U_t \otimes \tilde{H}_t$$

Simpler in design and in training

Seems to be more efficient in general, especially with less training data

LSTM

$$F_t = \sigma(W_{X_F} X_t + W_{Y_F} Y_{t-1} + B_F)$$

$$I_t = \sigma(W_{X_I} X_t + W_{Y_I} Y_{t-1} + B_I)$$

$$O_t = \sigma(W_{X_O} X_t + W_{Y_O} Y_{t-1} + B_O)$$

$$H_t = F_t \otimes H_{t-1} + I_t \otimes \tanh(W_{X_H} X_t + W_{Y_H} Y_{t-1} + B_H)$$

$$Y_t = O_t \otimes \tanh(H_t)$$

More weights to train

Seems to outperform GRU for applications with very long dependencies

Not clear when to use one or the other, as performances are comparable in many tasks

OUTLINE

- [Reminder] Neural networks 101
- Deep learning for time series:
 - Recurrent neural network (RNN)
 - Long short-term memory (LSTM)
 - Gated recurrent unit (GRU)
 - 1D convolutional neural network (1D-CNN)
- Transformers

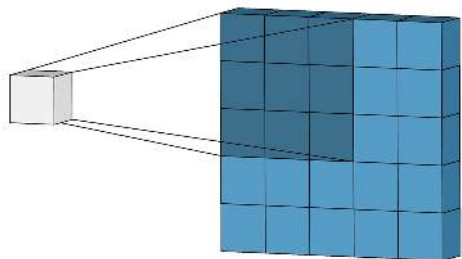


Deep learning for time series – 1D-CNN

└ Principle of convolution (in 2D)

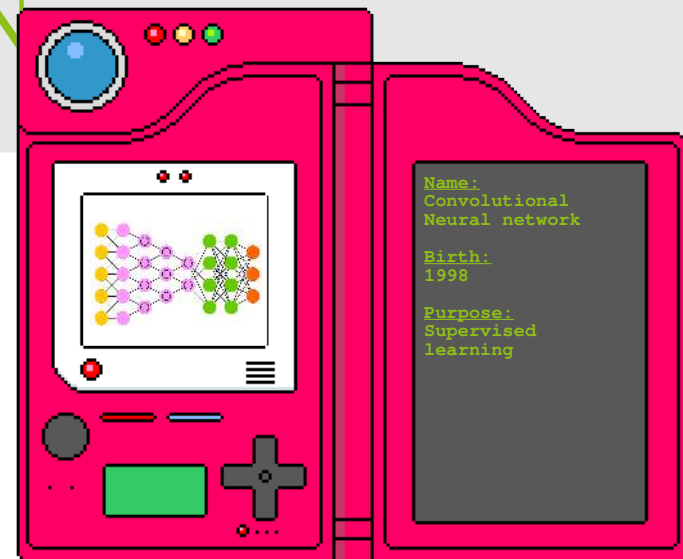
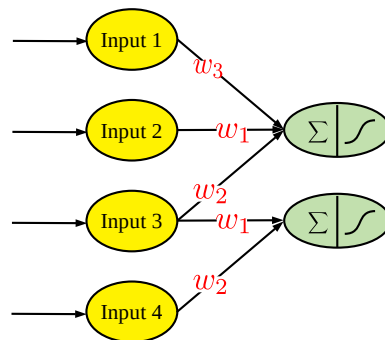
<https://kamaleshjicse.files.wordpress.com/2018/11/pokedex-kamalesh.gif>

<https://towardsdatascience.com/intuitively-understanding-convolutions-for-deep-learning-1f6f42faee1>



3_0	3_1	2_2	1	0
0_2	0_2	1_0	3	1
3_0	1_1	2_2	2	3
2	0	0	2	2
2	0	0	0	1

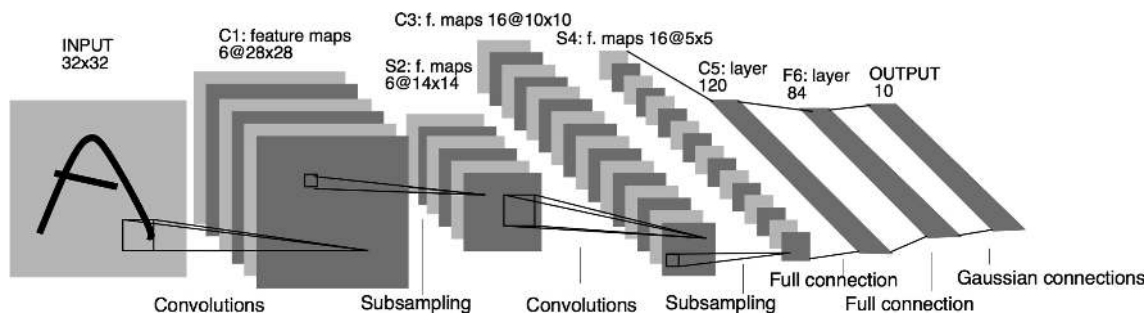
12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0



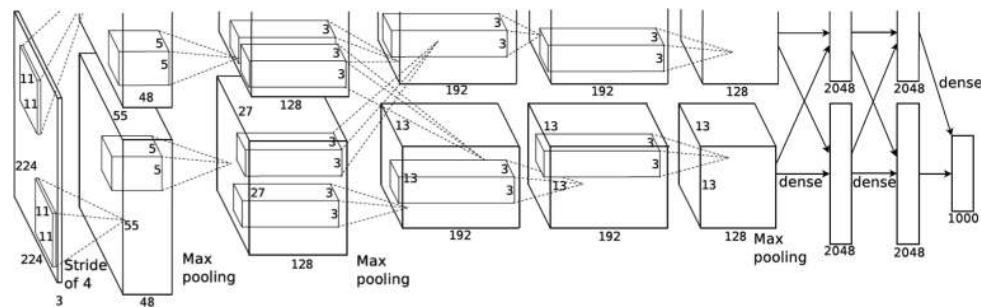
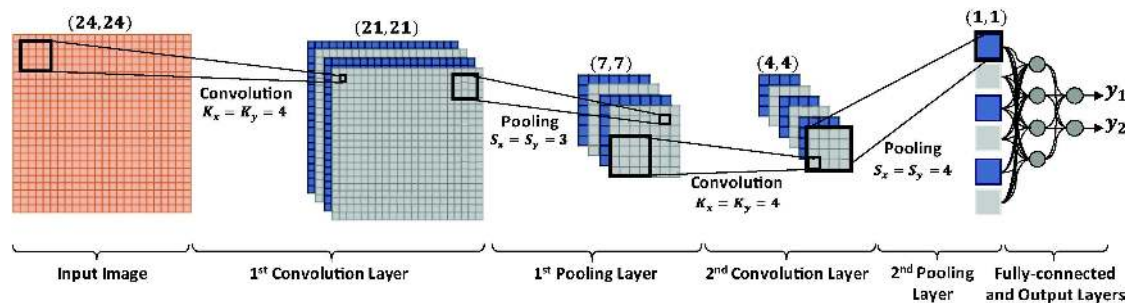
- Linear combination is replaced by convolution (captures locality of features)
- Some weights are shared (translation invariance)
- Ideal for data lying on Euclidean spaces
- Graph CNN extend this idea to irregular domains

Deep learning for time series – 1D-CNN

└ Assembling 2D convolutions to build CNNs



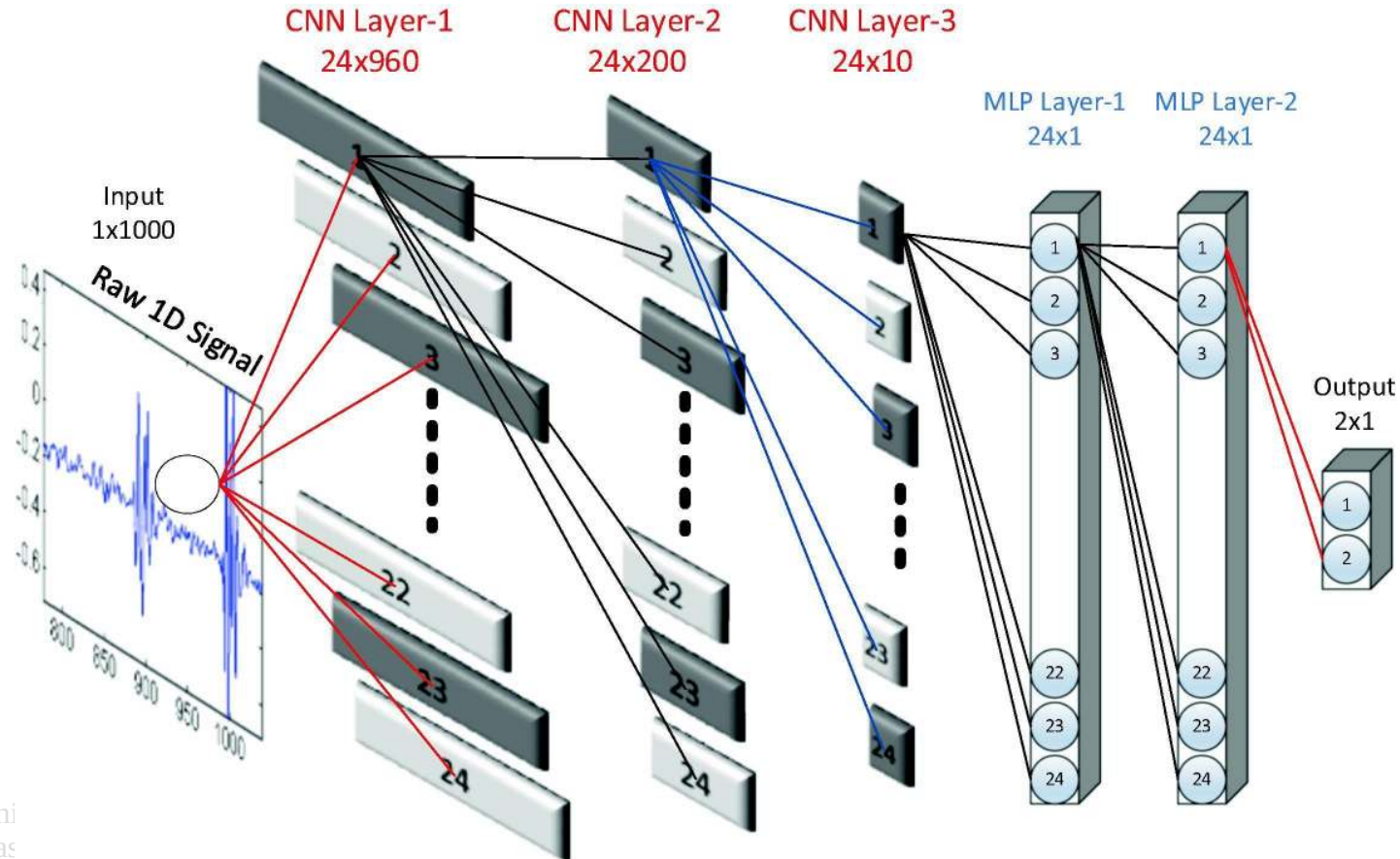
<https://www.sciencedirect.com/science/article/pii/S0888327020307846>



Deep learning for time series – 1D-CNN

└ It's basically the same in 1D

<https://www.sciencedirect.com/science/article/pii/S0888327020307846>



Deep learning for time series – 1D-CNN

└ You can mix everything!



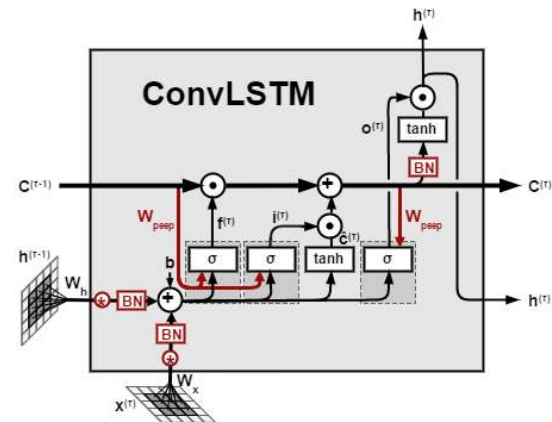
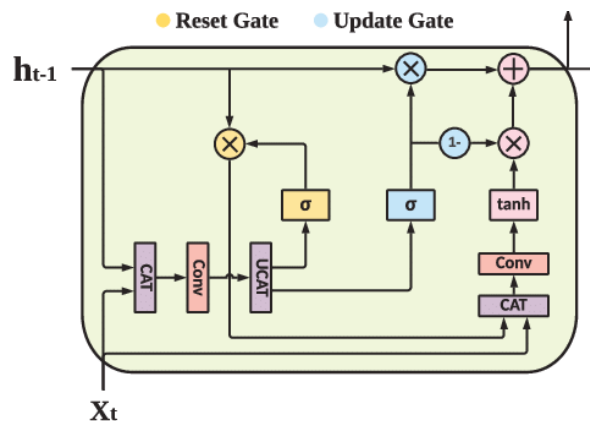
Bastien Padeloup
14/11/2022

<https://medium.com/neuronio/an-introduction-to-convlstm-55c9025563a7>

<https://arxiv.org/abs/2108.08404>

<https://www.leselestar.fr/actu-tv/maite-que-devient-l-ancienne-animatrice-de-la-cuisine-des-mousquetaires-352519>

<https://towardsdatascience.com/intuitively-understanding-convolutions-for-deep-learning-1f6f42faee1>



OUTLINE

- [Reminder] Neural networks 101
- Deep learning for time series:
 - Recurrent neural network (RNN)
 - Long short-term memory (LSTM)
 - Gated recurrent unit (GRU)
 - 1D convolutional neural network (1D-CNN)
- Transformers



Natural language processing

From Wikipedia, the free encyclopedia

Natural language processing (NLP) is a subfield of [linguistics](#), [computer science](#), and [artificial intelligence](#) concerned with the interactions between computers and human language, in particular how to program computers to process and analyze large amounts of [natural language](#) data. The goal is a computer capable of "understanding" the contents of documents, including the [contextual](#) nuances of the language within them. The technology can then accurately extract information and insights contained in the documents as well as categorize and organize the documents themselves.

Challenges in natural language processing frequently involve [speech recognition](#), [natural-language understanding](#), and [natural-language generation](#).

Transformers

└ Applications of NLP

<https://www.mygreatlearning.com/blog/trending-natural-language-processing-applications>
<https://www.analyticssteps.com/blogs/top-10-applications-natural-language-processing-nlp>

Marketing
Sentiment analysis
Recruitment
Text summarization
Survey analysis
Targeted advertising
Translation

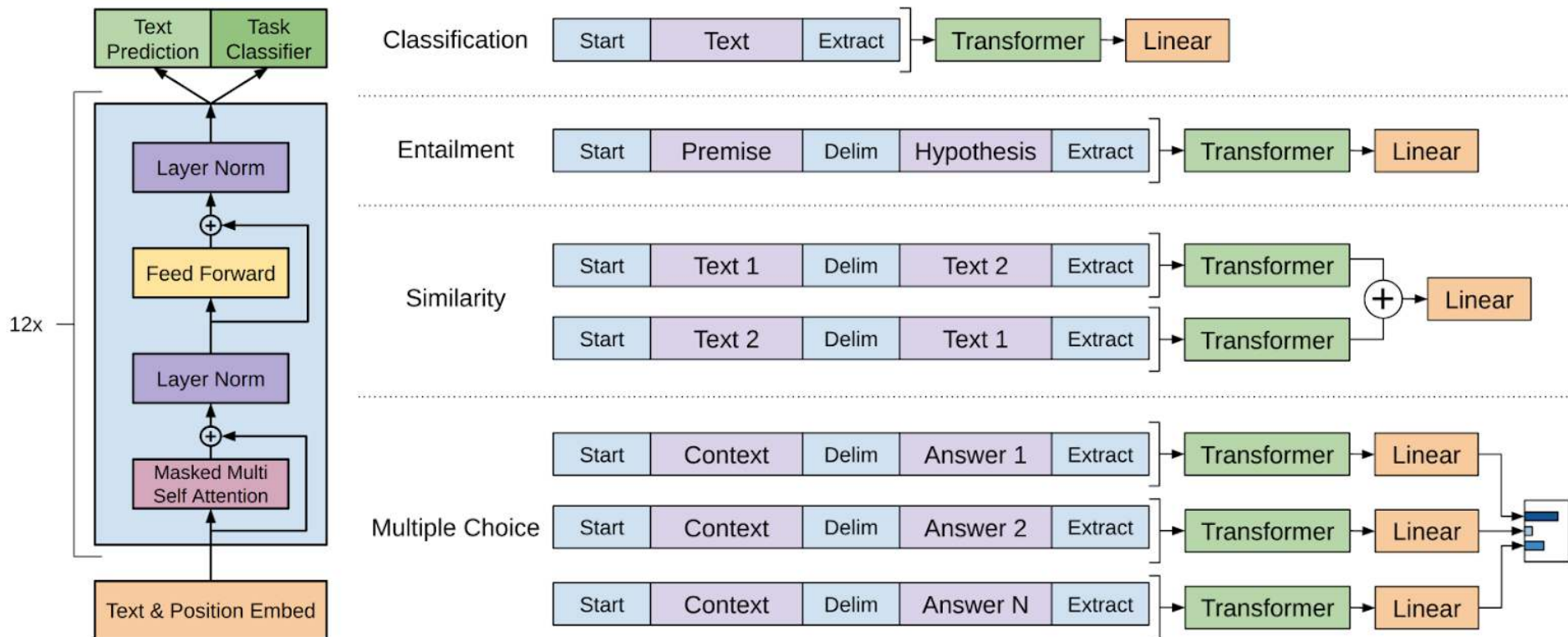
Speech recognition
Chatbots
Autocorrectors
Copywriting
Email filtering
Grammar check
...

Transformers

└ Some transformer architectures (1/2)

<https://france.devoteam.com/paroles-dexperts/lstm-transformers-gpt-bert-guide-des-principales-techniques-en-nlp>

GPT-3



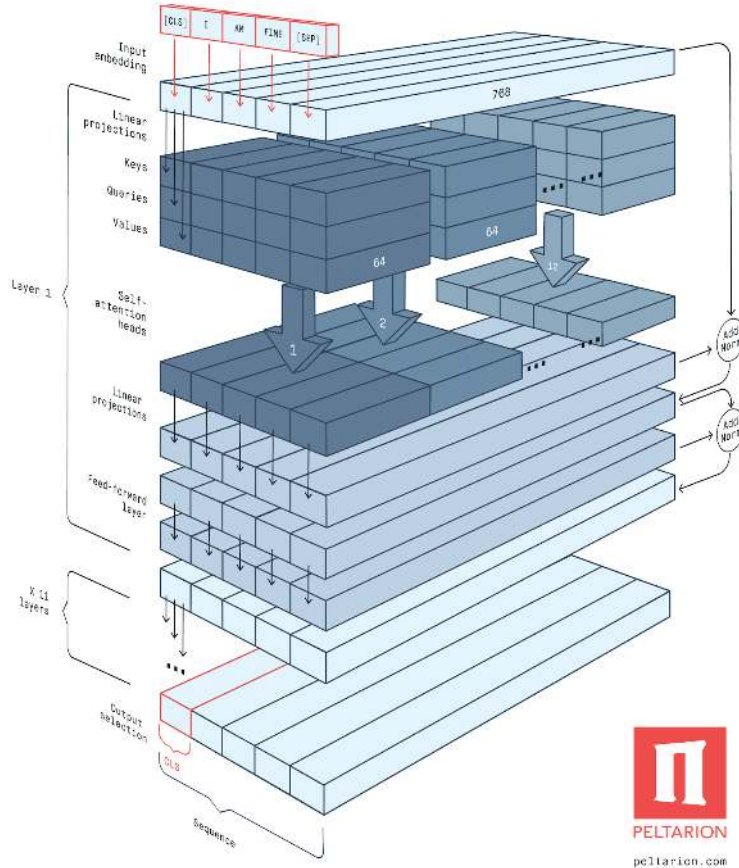
Transformers

Some transformer architectures (2/2)

<https://france.devoteam.com/paroles-dexperts/lstm-transformers-gpt-bert-guide-des-principales-techniques-en-nlp/>
<https://lbourdois.github.io/blog/nlp/Les-architectures-transformers/>

BERT

Etc.



Modèle	Description	Nb de paramètres	Taille du vocabulaire	Tokenizer utilisé
GPT-2	Transformer unidirectionnel développé par OpenAI utilisant uniquement la partie decodeur du transformer original.	4 versions : 124M, 355M, 774M ou 1.5 Mds	50K	BPE
BERT	Transformer bidirectionnel développé par Google utilisant uniquement la partie encodeur du transformer original.	Tiny : 4.4M, Mini : 11.3M, Small : 29.1M, Medium : 41.7M, Base : 110M, Large : 334M, XLarge : 1270M	30K	WordPiece
RoBERTa	Version optimisée de BERT développée par Facebook.	Base : 125M, Large : 355M	50K	BPE
CamemBERT	Modèle basé sur RoBERTa pré-entraîné sur des textes en français et développé par Facebook, l'INRIA et la Sorbonne	110M	32K	SentencePiece
FlauBERT	Modèle basé sur BERT pré-entraîné sur des textes en français et développé par l'Univ. Grenoble Alpes, le CNRS et l'Univ. Paris Diderot	small-cased : 54M, base-uncased : 137M, base-cased : 138M et large-cased : 373M	50K	FastBPE
BART	Modèle basé sur BART (voir plus bas) pré-entraîné sur des textes en français et développé par une équipe de l'Ecole Polytechnique	216 M (il existe aussi un mBART, un BART multilingue de 561 ou 866M de paramètres)	50K	SentencePiece
PAGNOL	Modèle basé sur GPT-2 pré-entraîné sur des textes en français et développé par LightOn	S : 124M, M : 355M, L : 773M, XL : 1.5 Md	50K	BPE
FrALBERT	Modèle basé sur ALBERT (voir plus bas) pré-entraîné sur des textes en français et développé par Qwant	12M	32K	SentencePiece
Cedille	Modèle basé sur GPT-J pré-entraîné sur des textes en français et développé par Coteries	6Mds	50K	BPE
Distil*	Versions distillées (plus petites donc plus rapides) de BERT, du GPT-2 et de RoBERTa développées par Huggingface.	66M pour DistilBERT, 82M pour DistilGPT2 et DistilRoBERTa	Identique au modèle original	Identique au modèle original
XLNet	Transformer bidirectionnel développé par Google et la Carnegie Mellon University avec pré-entraînement auto-supervisé	Identique à BERT	32K	SentencePiece

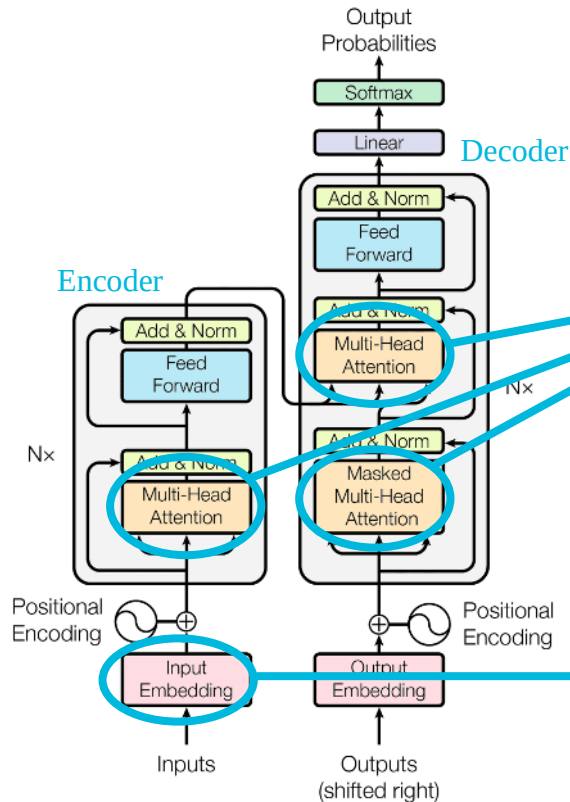
Transformers

└ The attention mechanism

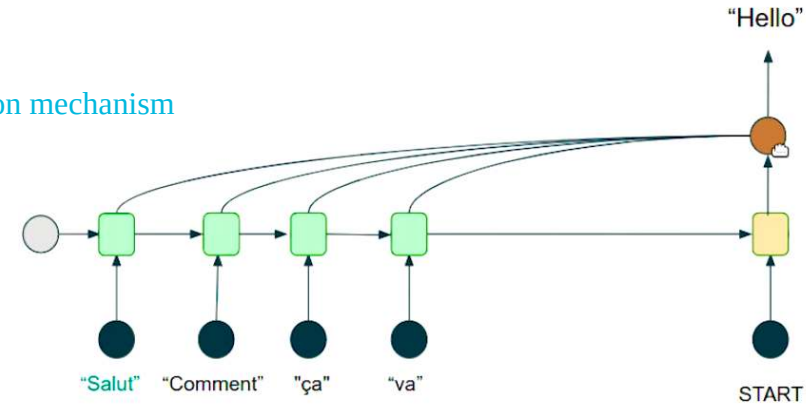
Generic transformer architecture

<https://arxiv.org/abs/1706.03762>

<https://www.youtube.com/watch?v=LALTmQhVkfU>



Attention mechanism



Embedding:

- * One-hot encoding (sometimes)
- * Embedding space (generally)

Transformers

└ Scaled dot product attention (1/3)

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

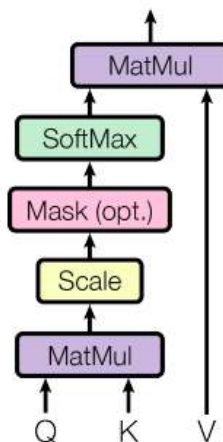
Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaiser@google.com

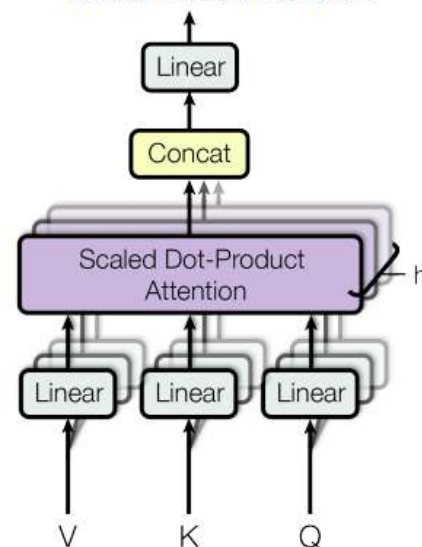
Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Scaled Dot-Product Attention



Multi-Head Attention



<https://arxiv.org/abs/1706.03762>
<https://www.youtube.com/watch?v=LALTmQhVkfU>

Transformers

└ Scaled dot product attention (2/3)

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

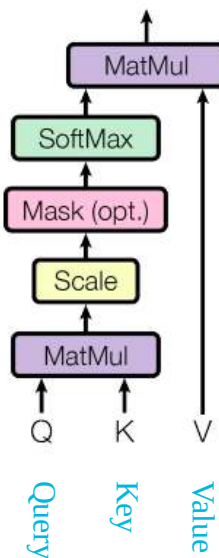
Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaier@google.com

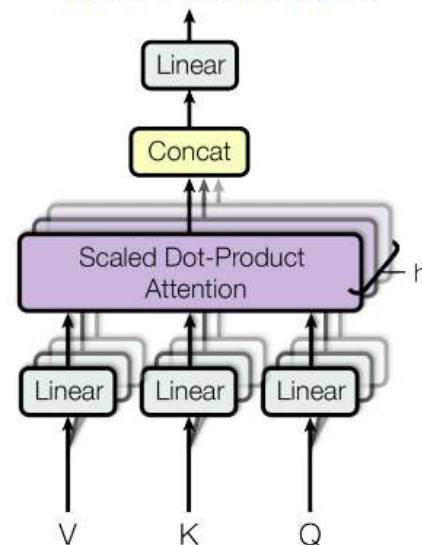
Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Scaled Dot-Product Attention



Multi-Head Attention



When trying to decode a **Query**,
how much does the **Value** associated with a
Key (in the encoder states) help for decoding?

Transformers

└ Scaled dot product attention (3/3)

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

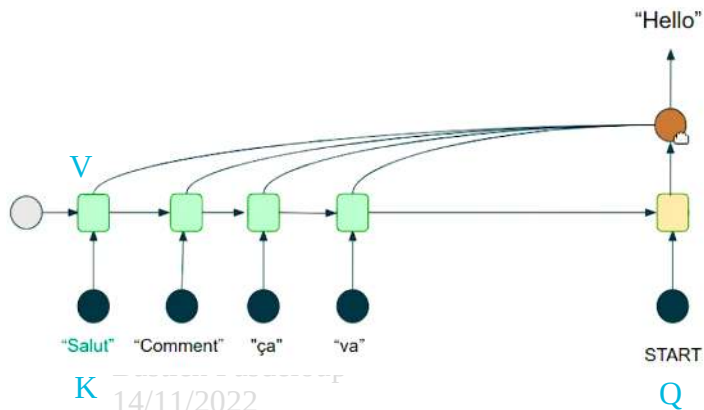
Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*
University of Toronto
aidan@cs.toronto.edu

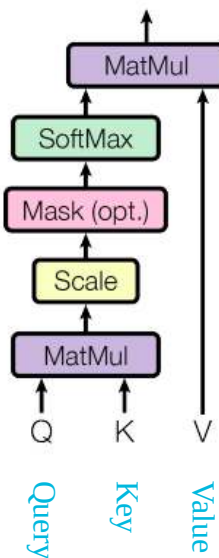
Lukasz Kaiser*
Google Brain
lukaszkaier@google.com

Illia Polosukhin*
illia.polosukhin@gmail.com



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

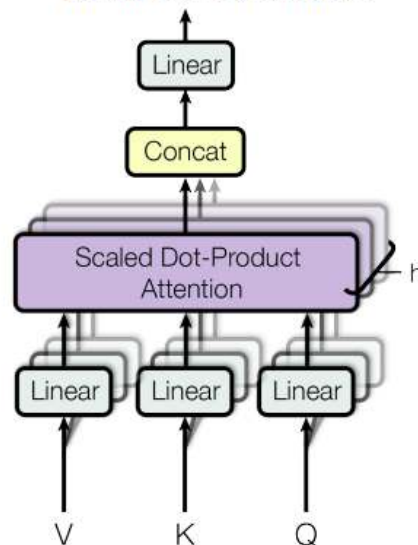
Scaled Dot-Product Attention



When trying to decode a Query,
how much does the Value associated with a
Key (in the encoder states) help for decoding?

<https://arxiv.org/abs/1706.03762>
<https://www.youtube.com/watch?v=LALTmQhVkfU>

Multi-Head Attention



Transformers

Self-attention illustrated

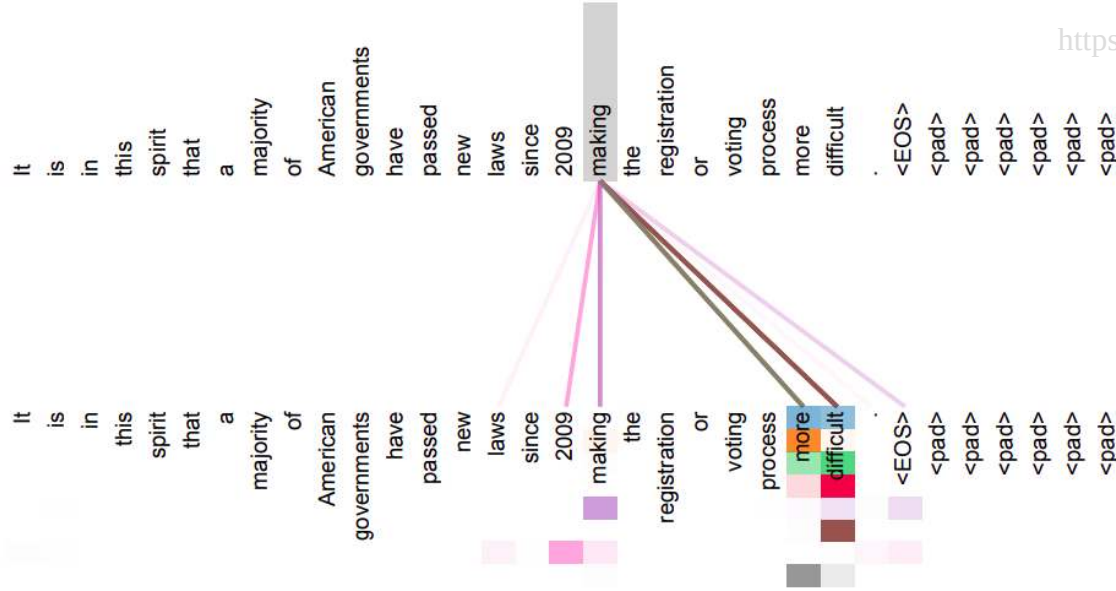
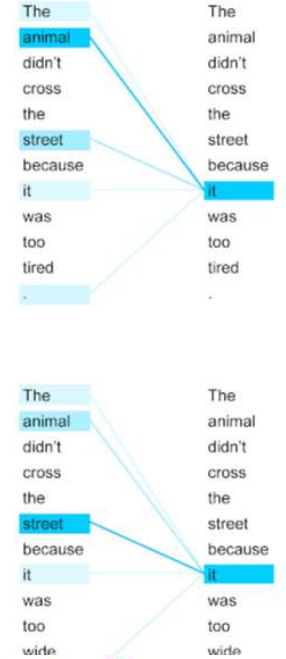


Figure 3: An example of the attention mechanism following long-distance dependencies in the encoder self-attention in layer 5 of 6. Many of the attention heads attend to a distant dependency of the verb 'making', completing the phrase 'making...more difficult'. Attentions here shown only for the word 'making'. Different colors represent different heads. Best viewed in color.

<https://arxiv.org/pdf/1706.03762.pdf>
<https://ai.googleblog.com/2017/08/transformer-novel-neural-network.html>



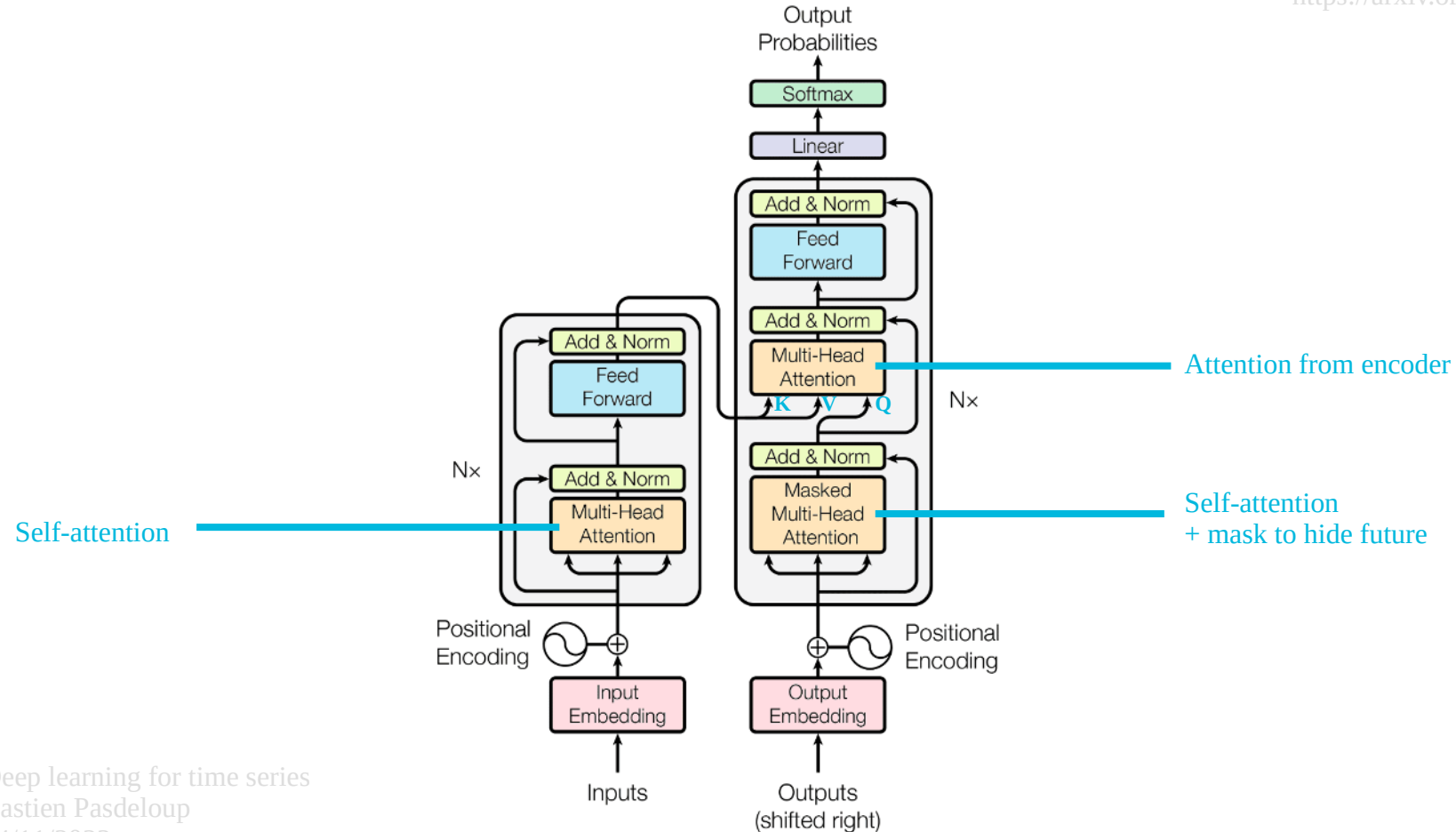
The animal didn't cross the street because it was too tired.
L'animal n'a pas traversé la rue parce qu'il était trop fatigué.

The animal didn't cross the street because it was too wide.
L'animal n'a pas traversé la rue parce qu'elle était trop large.

Transformers

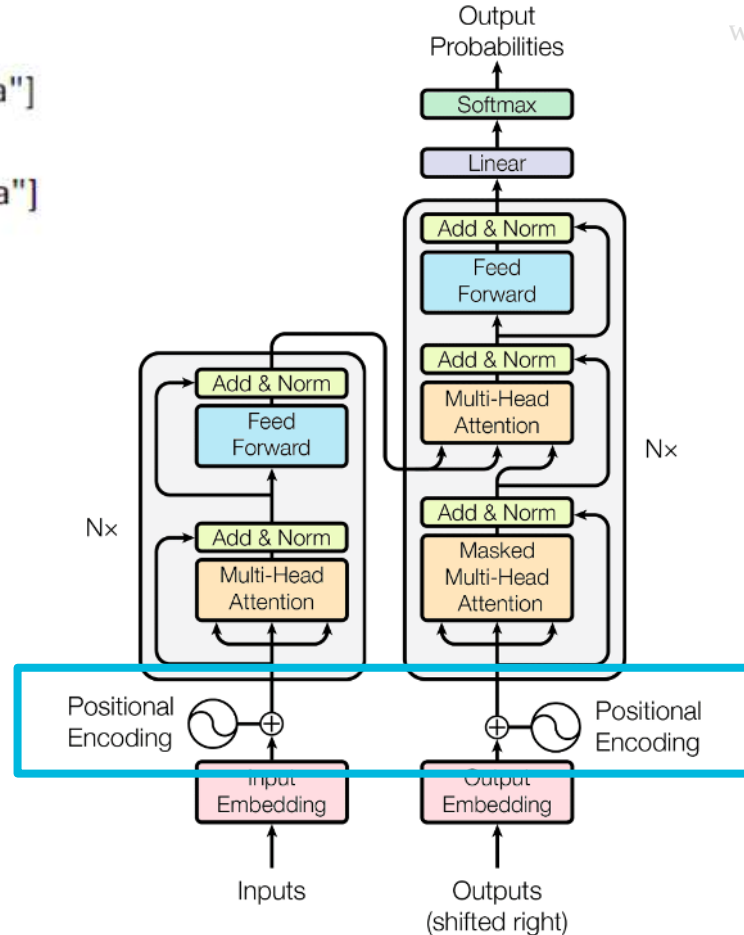
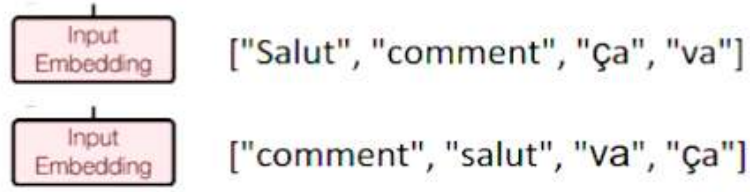
└ Attention at the center of transformers

<https://arxiv.org/abs/1706.03762>



Transformers

Positional encoding

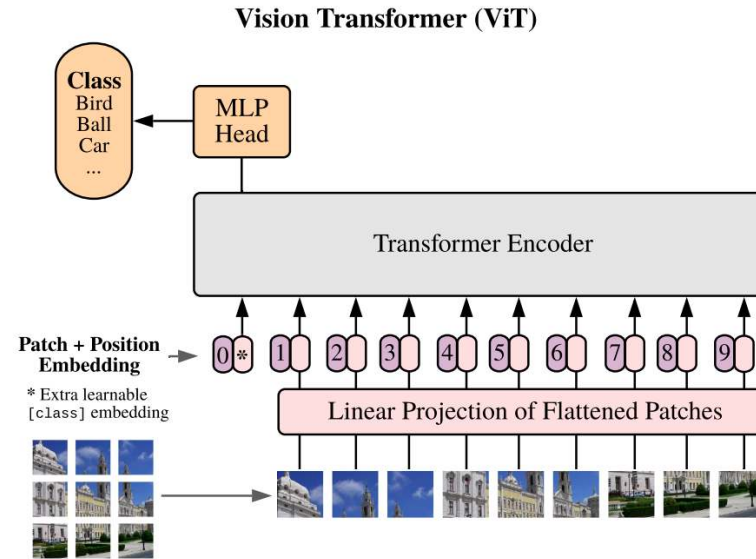
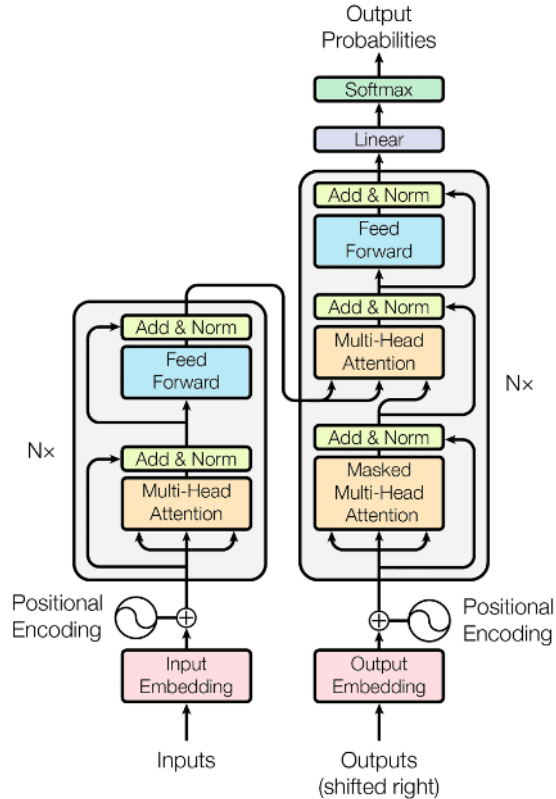


<https://arxiv.org/abs/1706.03762>
www.youtube.com/watch?v=LALTmQhVkfU

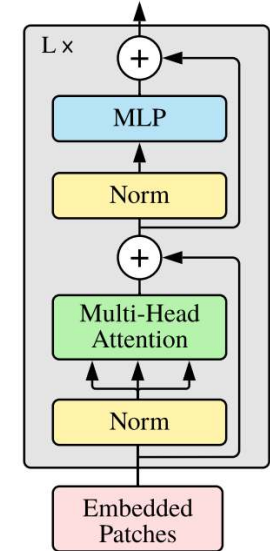
Transformers

Extension to images

<https://arxiv.org/abs/1706.03762>
<https://arxiv.org/abs/2010.11929>



Transformer Encoder



TAKE-HOME MESSAGES

Different models for different data types

Convolutions for locally coherent data

RNN etc. for time series

- [Reminder] Neural networks 101
- Deep learning for time series:
 - Recurrent neural network (RNN)
 - Long short-term memory (LSTM)
 - Gated recurrent unit (GRU)
 - 1D CNN (1D-CNN)
- Transformers



TAKE-HOME MESSAGES

The classical model for time series
Suffers from gradient vanishing

- [Reminder] Neural networks 101
- Deep learning for time series:
 - Recurrent neural network (RNN)
 - Long short-term memory (LSTM)
 - Gated recurrent unit (GRU)
 - 1D CNN (1D-CNN)
- Transformers



TAKE-HOME MESSAGES

Extension of the RNN

Solves the gradient vanishing problem

- [Reminder] Neural networks 101
- Deep learning for time series:
 - Recurrent neural network (RNN)
 - Long short-term memory (LSTM)
 - Gated recurrent unit (GRU)
 - 1D CNN (1D-CNN)
- Transformers



TAKE-HOME MESSAGES

Simplification of LSTM

- [Reminder] Neural networks 101
- Deep learning for time series:
 - Recurrent neural network (RNN)
 - Long short-term memory (LSTM)
 - Gated recurrent unit (GRU)
 - 1D CNN (1D-CNN)
- Transformers



TAKE-HOME MESSAGES

Convolutions can also be used for time series
CNN can be mixed with recurrent models

- [Reminder] Neural networks 101
- Deep learning for time series:
 - Recurrent neural network (RNN)
 - Long short-term memory (LSTM)
 - Gated recurrent unit (GRU)
 - 1D CNN (1D-CNN)
- Transformers



TAKE-HOME MESSAGES

State of the art in NLP and other domains

The future for series with temporal dependencies

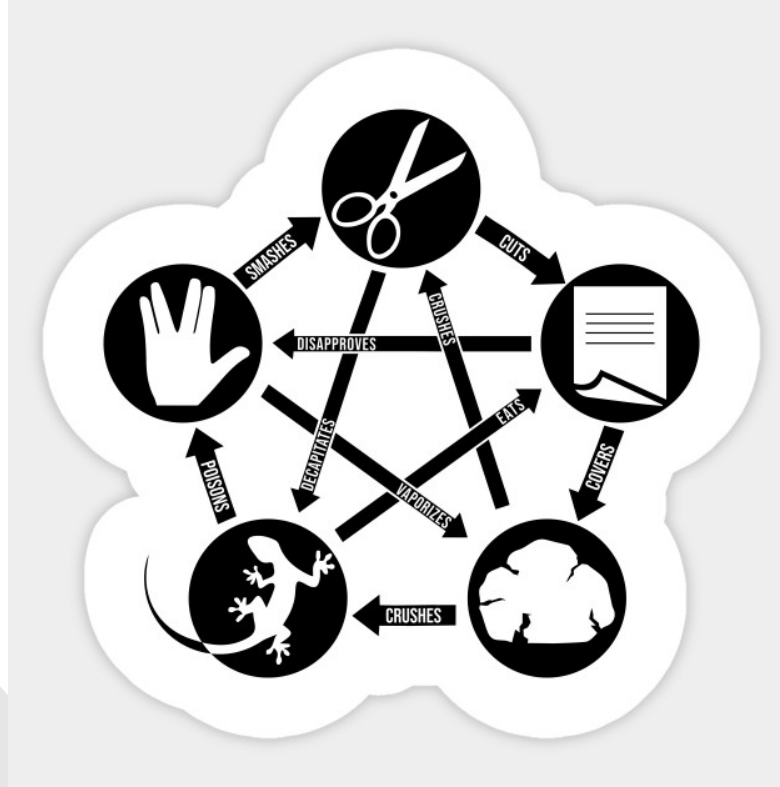
Attention is at the center of the models

- [Reminder] Neural networks 101
- Deep learning for time series:
 - Recurrent neural network (RNN)
 - Long short-term memory (LSTM)
 - Gated recurrent unit (GRU)
 - 1D CNN (1D-CNN)
- Transformers



PRACTICAL SESSION

➤ Today & tomorrow



DEEP LEARNING FOR TIME SERIES

